



南開大學
Nankai University

计算机学院
软件工程实验报告

第四次作业—编程设计

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2026 年 6 月 3 日

目 录

1 实验概述	3
1.1 题目说明	3
1.2 实验环境	3
2 代码设计与实现	3
2.1 总体结构	3
2.2 输入校验模块	4
2.3 暴力遍历算法	5
2.4 基于 profile 的优化动机	5
2.5 单调队列优化算法	6
2.6 数组模拟队列优化算法	7
2.7 示例运行入口	8
3 a) 编程规范遵守情况	9
3.1 参考规范	9
3.2 静态检查与代码审查	10
3.3 检查方式与结果	10
4 b) 算法可扩展性分析	11
4.1 已经实现的扩展能力	11
4.2 未来扩展方向	11
5 c) SRP 与 OCP 原则遵守情况	11
5.1 单一职责原则	11
5.2 开放-封闭原则	12
6 d) 错误与异常处理	12
6.1 异常场景	12
6.2 处理方式	13
7 e) 算法复杂度与性能分析	14
7.1 复杂度分析	14
7.2 性能分析原则	14
7.3 性能分析方法	15
7.4 性能分析结果	17
8 f) 单元测试	19
8.1 测试目标与被测单元	19
8.2 测试驱动与公共断言工具	19
8.3 单元测试基本概念与质量指标	20
8.4 辅助检查思路	21
8.5 单元测试内容划分	22
8.6 静态测试与动态测试	22

8.7	黑盒测试方法与用例设计	22
8.7.1	等价类划分	23
8.7.2	边界值分析	25
8.7.3	健壮性测试	27
8.7.4	错误推测法	28
8.8	白盒测试方法与用例设计	29
8.8.1	基于控制流的测试过程	29
8.8.2	流程图与控制流图	30
8.8.3	测试需求、覆盖标准和路径选择标准	32
8.8.4	六种代码覆盖标准	32
8.8.5	基本路径测试与循环测试	34
8.9	unittest 与 Mock 测试实现	37
8.10	测试用例总表	39
8.11	测试执行结果与覆盖率分析	41
8.12	缺陷分析与测试结论	42
9	实验总结	42

1 实验概述

1.1 题目说明

在本次实验中我们需要实现滑动窗口最大值问题：给定一个整数数组 `nums` 和一个窗口大小 `k`，窗口从数组最左侧开始，每次向右移动一位。实验目标是返回每一次窗口移动后窗口内的最大值组成的数组。例如：

```
nums=[1, 3, -1, -3, 5, 3, 6, 7], k=3
```

窗口最大值序列应为：

```
[3, 3, 5, 5, 6, 7]
```

作业要求不仅包括算法实现，还要求用 `pylint` 进行代码分析，用 `profile/cProfile` 进行性能分析，并用 `unittest` 完成单元测试。因此，本实验的重点不只是写出一个能运行的算法，还要从编程规范、错误处理、可扩展性、设计原则、复杂度和测试质量等角度说明实现过程。

1.2 实验环境

本次实验采用独立 `conda` 环境执行检查，避免不同 Python 版本或工具版本造成结果不一致。实际环境如下：

表 1: 实验环境

项目	版本或说明
操作系统	macOS
Python	3.11.15
代码规范检查	pylint 4.0.5
覆盖率统计	coverage.py 7.13.4
测试框架	Python 标准库 unittest
性能分析	Python 标准库 cProfile、pstats、time.perf_counter

2 代码设计与实现

2.1 总体结构

我们并没有直接把所有逻辑堆叠在入口函数中，而是将其拆成三个相对独立的文件：

- `sliding_window_max.py`: 包含输入校验、暴力遍历算法、单调队列算法、数组模拟队列算法和示例运行入口；
- `test_sliding_window_max.py`: 使用 `unittest` 对正常用例、边界用例和异常用例进行测试；
- `profile_sliding_window.py`: 构造固定规模测试数据，记录三种算法的运行时间，并输出 `profile` 统计结果。

这种拆分方式使核心算法、测试逻辑和性能分析逻辑互不混杂。算法文件只负责给出可复用函数，测试文件只负责验证行为，性能脚本只负责构造可复现实验并输出分析数据。三个可直接执行的 Python 文件均保留 `if __name__ == "__main__"` 入口，保证文件被导入时不会自动运行主流程。

2.2 输入校验模块

下面的代码是三个算法共用的输入校验函数，它位于算法调用的最前端，负责把非法输入拦截在核心计算之前。这样做的好处是后续无论增加新的算法版本，还是把当前算法封装进其他程序，都可以复用同一套边界规则。

Listing 1: 输入校验函数

```
1 def validate_window_input(nums: list[int], k: int) -> None:
2     """校验滑动窗口输入数据。
3
4     参数：
5         nums：用于滑动窗口计算的整数列表。
6         k：正整数窗口大小。
7
8     异常：
9         TypeError：当 nums 不是列表、k 不是整数，或 nums 中含非整数元素时抛出。
10        ValueError：当 nums 为空、k 非正数，或 k 大于数组长度时抛出。
11    """
12    if not isinstance(nums, list):
13        raise TypeError("nums 必须是整数列表。")
14
15    if not nums:
16        raise ValueError("nums 不能为空。")
17
18    if not isinstance(k, int) or isinstance(k, bool):
19        raise TypeError("k 必须是整数。")
20
21    if k <= 0:
22        raise ValueError("k 必须大于 0。")
23
24    if k > len(nums):
25        raise ValueError("k 不能大于 nums 的长度。")
26
27    for index, value in enumerate(nums):
28        # bool 是 int 的子类，需要单独排除，避免 True/False 被当作整数数据。
29        if not isinstance(value, int) or isinstance(value, bool):
30            value_type = type(value).__name__
31            raise TypeError(
32                f"nums[{index}] 必须是整数，实际类型为 {value_type}。"
33            )
```

这段代码是后面三个算法函数的共同前置步骤。每个算法进入核心循环前都会先完成输入校验，校验内容包括输入类型、空数组、窗口大小范围以及数组元素类型。特别地，我们在代码中显式排除了布尔值，因为在 Python 中 bool 是 int 的子类，如果不单独处理，True 和 False 会被误认为合法整数。

2.3 暴力遍历算法

暴力遍历版作为基准实现，思路是枚举每一个窗口，并对窗口切片调用 `max`。它的逻辑直接、容易检查，适合用于验证优化算法是否保持了相同结果。

Listing 2: 暴力遍历版滑动窗口最大值

```

1 def sliding_window_max_brute_force(nums: list[int], k: int) -> list[int]:
2     """使用暴力遍历法计算每个滑动窗口的最大值。
3
4     该函数逐个扫描窗口并计算窗口内最大值，适合作为正确性基准。
5     由于相邻窗口之间有大量重叠元素，暴力遍历会重复进行比较。
6
7     参数：
8         nums: 整数列表。
9         k: 正整数窗口大小。
10
11     返回：
12         每个滑动窗口最大值组成的列表。
13     """
14     validate_window_input(nums, k)
15
16     max_values = []
17     window_count = len(nums) - k + 1
18     for left_index in range(window_count):
19         right_index = left_index + k
20         max_values.append(max(nums[left_index:right_index]))
21
22     return max_values

```

这段代码承接前面的输入校验函数，然后计算窗口个数。窗口个数为 `len(nums) - k + 1`。每次循环中，左右下标划定当前窗口范围，`max` 得到该窗口最大值并追加到结果列表。该朴素算法为后续单调队列优化提供了一个清晰的正确性对照。

2.4 基于 profile 的优化动机

暴力遍历版实现后，我们先使用 `cProfile` 对固定规模输入进行分析，再决定优化方向。性能分析采用 `data_size=12000`、`window_size=120`，输出结果长度为 11881。暴力版 profile 结果中最关键的调用如下：

Listing 3: 暴力版 profile 暴露的主要瓶颈

```

47770 function calls in 0.014 seconds

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.003    0.003    0.014    0.014 sliding_window_max.py:54(sliding_window_max_brute_force)
11881   0.008    0.000    0.008    0.000 {built-in method builtins.max}

```

这说明暴力版的主要瓶颈并不在结果列表追加或输入校验，而在于调用 `max` 对每个窗口的重复扫描。由于相邻窗口只有一个元素离开、一个元素进入，大部分元素会在多个窗口中被反复比较，从而

导致性能问题。针对这一瓶颈，优化方向首先应从数据结构和算法入手，而不是只调整局部语句。单调队列能够保存仍可能成为最大值的候选下标，并在窗口移动时及时移除过期元素和较小候选，从而避免每个窗口重新扫描全部 k 个元素。

2.5 单调队列优化算法

单调队列版的核心思想是保存仍可能成为窗口最大值的下标。队列中的下标对应的数值保持非递增顺序，因此队首总是当前窗口的最大值下标。当窗口右移时，过期下标会从队首移除；当新元素进入窗口时，所有不大于新元素的队尾候选都会被移除。

Listing 4: 单调队列版滑动窗口最大值

```
1 def sliding_window_max_monotonic_queue(nums: list[int], k: int) -> list[int]:
2     """使用单调队列计算每个滑动窗口的最大值。
3
4     队列保存数组下标而不是直接保存数值。队列中下标对应的数值保持
5     非递增顺序，因此队首下标总是指向当前窗口最大值。
6
7     参数：
8         nums: 整数列表。
9         k: 正整数窗口大小。
10
11     返回：
12         每个滑动窗口最大值组成的列表。
13     """
14     validate_window_input(nums, k)
15
16     max_values = []
17     candidate_indexes: deque[int] = deque()
18
19     for current_index, current_value in enumerate(nums):
20         # 队首下标若已离开窗口，就不能再参与当前窗口最大值判断。
21         while candidate_indexes and candidate_indexes[0] <= current_index - k:
22             candidate_indexes.popleft()
23
24         # 新元素更大时，队尾较小候选不会再成为后续窗口最大值。
25         while (
26             candidate_indexes
27             and nums[candidate_indexes[-1]] <= current_value
28         ):
29             candidate_indexes.pop()
30
31         candidate_indexes.append(current_index)
32
33         if current_index >= k - 1:
34             max_values.append(nums[candidate_indexes[0]])
35
36     return max_values
```

这段代码同样先调用公共校验函数，然后使用 `deque` 保存候选下标。它与暴力版输出相同的结果列表，因此测试文件可以用同一组预期值同时验证多种算法。相比暴力版每个窗口重新扫描，单调队列复用了相邻窗口之间的比较结果，避免了大量重复计算。

2.6 数组模拟队列优化算法

单调队列版再次 profile 后，主要耗时已经不再集中于窗口最大值扫描，而是由输入校验和队列维护等常数开销构成。其中 `deque.append`、`deque.pop` 和 `deque.popleft` 都属于常数时间操作，但仍会产生方法调用开销。相关 profile 输出如下：

Listing 5: 单调队列版继续优化前的 profile 依据

```
Monotonic-queue cProfile top calls:
59886 function calls in 0.007 seconds

Ordered by: cumulative time
List reduced from 9 to 8 due to restriction <8>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.004    0.004    0.007    0.007 sliding_window_max.py:78(sliding_window_max_monotonic_queue)
      1   0.001    0.001    0.002    0.002 sliding_window_max.py:19(validate_window_input)
24003   0.001    0.000    0.001    0.000 {built-in method builtins.isinstance}
11881   0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
11893   0.000    0.000    0.000    0.000 {method 'pop' of 'collections.deque' objects}
12000   0.000    0.000    0.000    0.000 {method 'append' of 'collections.deque' objects}
     105   0.000    0.000    0.000    0.000 {method 'popleft' of 'collections.deque' objects}
      1   0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

这组数据说明，单调队列版已经消除了暴力版中 `max` 重复扫描的主要瓶颈，继续优化不再属于复杂度层面的改进，而是面向队列维护中方法调用次数的常数开销控制。基于此，本实验保留单调队列思想，另外实现一个数组模拟队列版：使用列表保存候选下标，再用 `head_index` 和 `tail_index` 表示队首和队尾，该版本的时间复杂度仍为 $O(n)$ ，但减少了部分队列方法调用。

Listing 6: 数组模拟队列版滑动窗口最大值

```
1 def sliding_window_max_array_queue(nums: list[int], k: int) -> list[int]:
2     """使用数组模拟队列计算每个滑动窗口的最大值。
3
4     该函数与单调队列版保持相同算法思想，但使用列表和首尾指针模拟队列，
5     减少 deque 方法调用带来的常数开销，便于进行同层级性能对比。
6
7     参数：
8         nums: 整数列表。
9         k: 正整数窗口大小。
10
11     返回：
12         每个滑动窗口最大值组成的列表。
13     """
14     validate_window_input(nums, k)
```



```

15
16 max_values = []
17 queue_indexes = [0] * len(nums)
18 head_index = 0
19 tail_index = 0
20
21 for current_index, current_value in enumerate(nums):
22     if (
23         head_index < tail_index
24         and queue_indexes[head_index] <= current_index - k
25     ):
26         head_index += 1
27
28     while (
29         head_index < tail_index
30         and nums[queue_indexes[tail_index - 1]] <= current_value
31     ):
32         tail_index -= 1
33
34     queue_indexes[tail_index] = current_index
35     tail_index += 1
36
37     if current_index >= k - 1:
38         max_values.append(nums[queue_indexes[head_index]])
39
40 return max_values

```

这段代码的衔接方式与单调队列版一致：先复用公共校验函数，再进入窗口扫描流程。不同之处在于，它不再调用 `deque` 的入队和出队方法，而是通过整数指针移动来维护候选下标区间。该优化不改变算法复杂度，这属于在确定算法结构之后对循环和数据结构常数开销的进一步控制。

2.7 示例运行入口

示例入口负责组织一次完整的命令行运行流程。程序使用题目给定数组和窗口大小作为固定输入，依次调用暴力遍历版、单调队列版和数组模拟队列版函数，并将三组结果输出到标准输出。入口函数只承担调度与展示职责，核心计算仍由前面的算法函数完成，因此同一组算法函数还可以被单元测试和性能分析脚本直接复用。

Listing 7: 示例运行入口

```

1 def run_demo() -> None:
2     """运行题目给定样例并输出三种算法的结果。"""
3     brute_force_result = sliding_window_max_brute_force(
4         DEMO_NUMBERS, DEMO_WINDOW_SIZE
5     )
6     monotonic_queue_result = sliding_window_max_monotonic_queue(
7         DEMO_NUMBERS, DEMO_WINDOW_SIZE
8     )
9     array_queue_result = sliding_window_max_array_queue(

```

```
10     DEMO_NUMBERS, DEMO_WINDOW_SIZE
11 )
12
13 print(f"nums = {DEMO_NUMBERS}")
14 print(f"k = {DEMO_WINDOW_SIZE}")
15 print(f"brute_force = {brute_force_result}")
16 print(f"monotonic_queue = {monotonic_queue_result}")
17 print(f"array_queue = {array_queue_result}")
18
19
20 def main(argv: list[str] | None = None) -> int:
21     """执行命令行示例入口。
22
23     参数:
24         argv: 命令行参数列表。本实验示例不需要额外命令行参数。
25
26     返回:
27         程序退出状态码, 0 表示正常结束。
28     """
29     _ = argv
30     run_demo()
31     return 0
32
33
34 if __name__ == "__main__": # pragma: no cover
35     raise SystemExit(main(sys.argv[1:]))
```

该入口位于公共校验函数和三个算法函数之后, 衔接了“输入样例、算法调用、结果输出”三个步骤。实际运行时, 三个版本均得到 [3, 3, 5, 5, 6, 7], 与题目样例的预期结果相同。后续在单元测试环节中我们将在更多正常、边界和异常输入上继续验证程序行为。

3 a) 编程规范遵守情况

3.1 参考规范

代码参考 Python PEP8 编程规范和 Google Python Style Guide 中的通用编码建议, 并结合 Python 程序模板、注释、命名、语句和导入规则进行编写, 再使用 pylint 的静态检查规则进行验证。具体执行时主要关注以下方面:

- 可直接执行的文件包含 `#!/usr/bin/env python3`、 `-*- coding: utf-8 -*-` 和作者信息;
- 主流程放入 `main(argv)` 或专门入口函数中, 并使用 `if __name__ == "__main__"` 控制直接执行行为;
- 缩进统一使用 4 个空格, 不使用制表符;
- 函数名、变量名采用小写加下划线形式, 例如 `sliding_window_max_monotonic_queue`;
- 模块、函数、测试类均编写中文文档字符串, 说明功能、参数、返回值或测试目的;

- 注释只解释容易误解的设计原因，例如为什么要排除布尔值，不机械重复代码语句本身；
- 导入语句集中放在文件开头，按标准库和项目模块分组，避免使用 `import *`；
- 函数之间保留空行，逻辑块之间使用适当空行分隔；
- 不在语句结尾添加分号，主体代码行宽控制在 79 个字符以内。

3.2 静态检查与代码审查

静态质量检查由自动化工具和人工检查两部分组成。自动化部分使用 `pylint` 检查代码风格、常见错误和潜在问题；人工检查则围绕接口定义、程序流程、异常处理、边界条件和性能风险进行复核。

表 2: 代码审查检查点

检查类别	检查重点	落实情况
编程规范	命名、缩进、注释、导入、语句格式	使用 PEP8 风格、中文文档字符串、4 空格缩进、无 <code>import *</code>
程序流程	循环结束条件、分支条件、主入口控制	窗口循环使用确定边界，脚本入口使用 <code>if __name__ == "__main__"</code> 控制
方法接口	参数类型、返回值、调用关系	三个公开函数签名固定，算法函数返回 <code>list[int]</code> ，校验函数不返回值
边界校验	数组长度、窗口大小、元素类型	集中在 <code>validate_window_input</code> 中处理，算法函数共享该校验逻辑
异常处理	非法输入、错误路径、异常类型	对非列表、空数组、非法 <code>k</code> 、非整数元素和布尔值分别抛出明确异常
性能风险	大规模输入、重复计算、数据结构选择	暴力版保留为基准，单调队列减少重复扫描，数组模拟队列用于比较常数开销

3.3 检查方式与结果

规范检查命令如下：

```
python -m pylint --persistent=n sliding_window_max.py test_sliding_window_max.py
profile_sliding_window.py
```

参数 `--persistent=n` 用于关闭 `pylint` 的缓存写入，使检查输出只反映代码质量本身。实际输出如下：

Listing 8: pylint 评测结果

```
-----
Your code has been rated at 10.00/10
```

从结果看，本次三个 Python 文件的 `pylint` 评分为 10.00/10。因此，可以认为当前实现符合本实验采用的 PEP8、程序模板与 `pylint` 检查要求。

4 b) 算法可扩展性分析

4.1 已经实现的扩展能力

在本实验中，我们在实现题目基本功能之外，还加入了若干便于扩展的设计。首先，实验代码没有只保留一种固定写法，而是提供了暴力遍历版、单调队列版和数组模拟队列版三个算法函数。三个函数解决的是同一个问题，但实现思路不同，便于在小规模数据、调试场景和性能场景之间进行对比。

其次，我们把输入校验从算法主体中抽离出来，由 `validate_window_input` 这个函数统一完成。这样每个算法函数不需要重复编写相同的类型判断和边界判断，后续如果输入规则发生变化，也只需要把修改范围集中在校验函数和相关测试用例中即可。

在当前实现中，除上述两点外，我们还设计了以下几个比较明确的扩展点：

- 三个算法函数使用统一函数签名 (`nums: list[int], k: int`) -> `list[int]`，后续增加其他算法时可以保持调用方式一致；
- 性能分析脚本单独编写，算法文件不依赖性能实验的数据规模；
- 单元测试文件单独编写，后续增加算法版本时，可以复用现有测试用例验证其正确性；
- 不同算法返回格式保持一致，便于在同一组测试数据和性能数据下进行对比。

需要说明的是，这些设计并没有改变题目本身的输入和输出要求，程序仍然处理整数数组的滑动窗口最大值问题，只是在代码结构上更方便后续增加新算法或调整功能。

4.2 未来扩展方向

若后续需求发生变化，我们也可以接着从数据、功能和软件构建这几个角度继续对程序进行扩展。

在数据层面，可以考虑支持浮点数数组、可比较对象或流式输入。当前实现严格限定整数数组，主要是为了贴合题目定义；如果后续要支持浮点数的话，需要修改校验函数，使其接受 `int` 与 `float`，并补充浮点数相关测试。

在功能层面，可以扩展为同时求窗口最大值、最小值、平均值或中位数。其中最大值和最小值可以复用单调队列思想，只需要调整比较方向即可；平均值可以通过滑动累加或前缀和来实现；中位数则需要引入双堆或有序容器，而不能直接套用当前最大值算法。新增这些功能时，原有最大值函数不需要修改，只需要新增对应函数。

在软件构建层面，可以将多个算法函数注册到统一调度表中，用字符串参数选择算法，例如 `brute_force` 或 `monotonic_queue`，这样的话就可以在不修改调用方主要流程的情况下增加新算法。

5 c) SRP 与 OCP 原则遵守情况

5.1 单一职责原则

单一职责原则要求一个模块或函数只承担一个明确原因引起的变化。在本实验中我们按不同职责对代码进行了拆分，没有把输入校验、算法计算、示例输出、测试和性能分析等模块全部混在一起。

在核心算法文件中，`validate_window_input` 只负责对输入进行合法性校验，例如检查 `nums` 是否为列表、`k` 是否为整数、窗口大小是否越界、数组元素是否为整数等，它并不参与窗口最大值的具体计算。

然后三个算法函数分别负责不同实现方式：`sliding_window_max_brute_force` 负责暴力版算法，`sliding_window_max_monotonic_queue` 负责单调队列优化版算法，`sliding_window_max_array_queue` 负责数组模拟队列版算法，每个函数只表达一种算法思路，便于单独阅读、测试和比较。

除核心算法外，`run_demo` 只负责题目示例输出，`test_sliding_window_max.py` 只负责对三个程序的行为验证，`profile_sliding_window.py` 只负责性能实验。这样的拆分则使得对于这些文件的维护范围比较明确：当异常规则变化时，我们主要修改校验函数和测试用例；当算法优化时，则主要修改对应算法函数；当性能分析数据规模变化时，也只需要修改性能脚本中的常量即可。这降低了修改范围，也降低了引入新错误的概率。

5.2 开放-封闭原则

开放-封闭原则要求软件实体对扩展开放、对修改封闭。本实验中的基础部分主要包括输入校验函数、统一算法接口和已有测试用例。后续扩展功能时，可以优先通过新增函数或新增脚本的方式完成，而不是直接修改已经验证过的算法主体。

例如，若新增堆实现版本，可以直接增加一个 `sliding_window_max_heap` 函数，并继续调用同一个校验函数；若新增命令行参数解析，可以让入口函数调用现有算法，而不修改已通过测试的算法主体；若新增性能对比算法，只需要在 `profile_sliding_window.py` 中追加算法函数对象，不需要改变已有算法的逻辑。

因此，`validate_window_input`、统一函数签名和既有测试用例都属于后续扩展时可以复用、原则上不应频繁修改的基础部分。不过，当前实现还不是完全自动注册的插件式结构，新增算法后，仍然需要手动把新函数加入测试和性能分析脚本中，但我认为对于本次实验的规模来说，这种方式比较直接，也更容易检查。

6 d) 错误与异常处理

6.1 异常场景

本实验处理的错误与异常场景如表 3 所示。

表 3: 异常处理场景

序号	异常场景	异常类型	处理说明
1	<code>nums</code> 不是列表	<code>TypeError</code>	输入容器类型不符合题目约定，直接拒绝
2	<code>nums</code> 为空列表	<code>ValueError</code>	无法形成任何窗口，因此拒绝
3	<code>k</code> 不是整数	<code>TypeError</code>	窗口大小必须是整数位移单位
4	<code>k <= 0</code>	<code>ValueError</code>	窗口大小必须为正数
5	<code>k > len(nums)</code>	<code>ValueError</code>	窗口不能大于数组长度
6	<code>nums</code> 中含非整数元素	<code>TypeError</code>	与整数数组题意不符
7	<code>nums</code> 中含布尔值	<code>TypeError</code>	避免 Python 将布尔值当作整数处理

6.2 处理方式

Listing 9: 输入校验函数

```

1 def validate_window_input(nums: list[int], k: int) -> None:
2     """校验滑动窗口输入数据。
3
4     参数:
5         nums: 用于滑动窗口计算的整数列表。
6         k: 正整数窗口大小。
7
8     异常:
9         TypeError: 当 nums 不是列表、k 不是整数, 或 nums 中含非整数元素时抛出。
10        ValueError: 当 nums 为空、k 非正数, 或 k 大于数组长度时抛出。
11    """
12    if not isinstance(nums, list):
13        raise TypeError("nums 必须是整数列表。")
14
15    if not nums:
16        raise ValueError("nums 不能为空。")
17
18    if not isinstance(k, int) or isinstance(k, bool):
19        raise TypeError("k 必须是整数。")
20
21    if k <= 0:
22        raise ValueError("k 必须大于 0。")
23
24    if k > len(nums):
25        raise ValueError("k 不能大于 nums 的长度。")
26
27    for index, value in enumerate(nums):
28        # bool 是 int 的子类, 需要单独排除, 避免 True/False 被当作整数数据。
29        if not isinstance(value, int) or isinstance(value, bool):
30            value_type = type(value).__name__
31            raise TypeError(
32                f"nums[{index}] 必须是整数, 实际类型为 {value_type}。"
33            )

```

异常处理主要集中在输入校验函数中, 而不是分散在三个算法实现内部。这样可以保证三个算法函数在面对非法输入时采用一致的处理方式。对于函数外部的调用方来说, 异常信息能够直接指出输入错误的原因, 比如 `TypeError`、`ValueError` 等等; 而对于算法实现本身来说, 完成输入校验后, 后续核心计算逻辑就可以默认输入已经满足题目约束, 不需要在循环中反复处理类型和边界问题, 从而使代码更简洁。

7 e) 算法复杂度与性能分析

7.1 复杂度分析

设数组长度为 n ，窗口大小为 k 。

暴力遍历版共有 $n - k + 1$ 个窗口，每个窗口需要扫描 k 个元素求最大值，因此时间复杂度为：

$$O((n - k + 1)k)$$

当 k 与 n 同阶时，可近似看作 $O(nk)$ 。暴力版额外空间主要来自输出数组，若不计输出数组，辅助空间为 $O(1)$ ；若计入输出结果，空间复杂度为 $O(n - k + 1)$ 。

单调队列版中，每个元素下标最多入队一次、出队一次。虽然循环中有 `while`，但所有弹出操作在整个数组范围内总次数不超过 n 。因此时间复杂度为：

$$O(n)$$

队列中最多保存 k 个候选下标，辅助空间复杂度为 $O(k)$ ；若计入输出数组，总空间复杂度为 $O(k + n - k + 1)$ ，通常记作 $O(n)$ 。

数组模拟队列版与单调队列版保持相同的单调队列思想，每个下标同样最多进入候选区间一次、离开候选区间一次，因此时间复杂度仍为 $O(n)$ 。该版本预先分配长度为 n 的下标数组，辅助空间复杂度为 $O(n)$ ；若只从算法思想看，它没有继续降低复杂度，而是通过减少 `deque` 方法调用来降低常数开销。

7.2 性能分析原则

性能分析遵循先保证正确性、再定位瓶颈、最后比较优化前后效果的顺序。本题先保留暴力遍历版作为基准，通过 `profile` 发现重复调用 `max` 是主要瓶颈，再使用单调队列改进数据结构和算法复杂度。在单调队列已经达到 $O(n)$ 后，继续分析其常数开销，并加入数组模拟队列版进行同层级对比。这样可以在同一输入上比较三种实现的结果和耗时，避免只追求速度而破坏功能正确性。

表 4: 性能分析原则与落实情况

原则	含义	落实情况
正确性优先	优化不能改变程序输出	性能脚本先比较三种算法输出，结果不一致时立即终止
先找瓶颈	通过测试和 <code>profile</code> 数据判断主要耗时位置	使用 <code>cProfile</code> 按累计耗时排序，观察暴力版中 <code>max</code> 的重复调用
先算法后代码	优先改进数据结构和算法，再考虑局部语句优化	将重复扫描窗口改为维护单调队列，复杂度由 $O(nk)$ 降为 $O(n)$
固定测试数据	测试数据应可复现，便于前后对比	使用固定随机种子、固定数组规模和固定窗口大小
前后评测	优化前后都需要运行并记录性能结果	同时输出暴力版、单调队列版、数组模拟队列版耗时和加速比
谨慎解释结果	单次时间受机器状态影响，不能夸大结论	结果分析只说明同一环境下单调队列系列实现明显快于暴力版，不扩展为绝对性能结论

7.3 性能分析方法

性能脚本使用固定随机种子生成整数数组，保证每次实验数据一致。脚本先分别测量三种算法的运行时间，再比较三个输出数组是否完全一致，最后用 `cProfile` 输出累计时间排序的函数调用统计。

本实验执行性能分析时使用的命令如下：

```
python profile_sliding_window.py
```

该命令会运行性能分析脚本。脚本内部调用 `cProfile.Profile()`，分别记录暴力遍历版、单调队列版和数组模拟队列版的函数调用情况，因此后续输出既包含整体耗时，也包含按累计耗时排序的 `profile` 统计结果。

Listing 10: 性能数据构造与 `profile` 封装

```

1 def build_profile_data(size: int = DATA_SIZE) -> list[int]:
2     """构造可复现的整数测试数据。"""
3     random.seed(RANDOM_SEED)
4     return [random.randint(-10_000, 10_000) for _ in range(size)]
5
6
7 def measure_algorithm(
8     algorithm: Callable[[list[int], int], list[int]],
9     nums: list[int],
10    k: int,
11 ) -> tuple[list[int], float]:
12     """运行一次算法，并返回计算结果和耗时。"""
13     start_time = time.perf_counter()
14     result = algorithm(nums, k)
15     elapsed_time = time.perf_counter() - start_time
16     return result, elapsed_time
17
18
19 def profile_algorithm(
20     algorithm: Callable[[list[int], int], list[int]],
21     nums: list[int],
22     k: int,
23 ) -> str:
24     """返回按累计耗时排序的 cProfile 文本结果。"""
25     profiler = cProfile.Profile()
26     profiler.enable()
27     algorithm(nums, k)
28     profiler.disable()
29
30     output_buffer = io.StringIO()
31     stats = pstats.Stats(profiler, stream=output_buffer)
32     stats.strip_dirs().sort_stats("cumtime").print_stats(8)
33     return output_buffer.getvalue()

```

这段代码负责建立性能实验的公共工具层。它与算法文件的衔接方式是：接收算法函数对象、输

入数组和窗口大小，然后返回运行结果、耗时或 `cProfile` 文本。因此性能脚本可以比较不同算法，而不需要把性能统计代码写进算法函数内部。

Listing 11: 性能实验执行流程

```

1 def run_profile() -> None:
2     """执行三种算法的计时和 cProfile 分析。"""
3     nums = build_profile_data()
4
5     brute_force_result, brute_force_time = measure_algorithm(
6         sliding_window_max_brute_force, nums, WINDOW_SIZE
7     )
8     monotonic_queue_result, monotonic_queue_time = measure_algorithm(
9         sliding_window_max_monotonic_queue, nums, WINDOW_SIZE
10    )
11    array_queue_result, array_queue_time = measure_algorithm(
12        sliding_window_max_array_queue, nums, WINDOW_SIZE
13    )
14
15    if brute_force_result != monotonic_queue_result:
16        raise RuntimeError("暴力版与单调队列版输出不一致。")
17
18    if brute_force_result != array_queue_result:
19        raise RuntimeError("暴力版与数组模拟队列版输出不一致。")
20
21    # 只有三种算法结果一致时，性能比较才具有实际意义。
22    monotonic_speedup = brute_force_time / monotonic_queue_time
23    array_speedup = brute_force_time / array_queue_time
24    array_speed_ratio = monotonic_queue_time / array_queue_time
25    print(f"data_size: {DATA_SIZE}")
26    print(f>window_size: {WINDOW_SIZE}")
27    print(f"result_length: {len(array_queue_result)}")
28    print(f"brute_force_time_seconds: {brute_force_time:.6f}")
29    print(f"monotonic_queue_time_seconds: {monotonic_queue_time:.6f}")
30    print(f"array_queue_time_seconds: {array_queue_time:.6f}")
31    print(f"monotonic_speedup_vs_brute_force: {monotonic_speedup:.2f}x")
32    print(f"array_speedup_vs_brute_force: {array_speedup:.2f}x")
33    print(f"array_speed_ratio_vs_monotonic_queue: {array_speed_ratio:.2f}x")
34    print()
35
36    print("Brute-force cProfile top calls:")
37    print(
38        profile_algorithm(
39            sliding_window_max_brute_force, nums, WINDOW_SIZE
40        ).strip()
41    )
42    print()
43    print("Monotonic-queue cProfile top calls:")
44    print(
45        profile_algorithm(

```

```

46     sliding_window_max_monotonic_queue, nums, WINDOW_SIZE
47     ).strip()
48 )
49 print()
50 print("Array-queue cProfile top calls:")
51 print(
52     profile_algorithm(
53         sliding_window_max_array_queue, nums, WINDOW_SIZE
54     ).strip()
55 )

```

这段代码则负责完整的性能实验流程：先构造同一组输入数据，再运行三种算法，随后检查三者结果是否一致。如果结果不一致，脚本会抛出 `RuntimeError`，避免在错误结果上继续讨论性能。只有确保正确性一致后，脚本才输出耗时、加速比和 `profile` 统计信息。这样的顺序体现了先验证正确性、再评估优化效果的原则，避免为了速度牺牲结果正确性。

7.4 性能分析结果

实际运行参数为 `data_size=12000`、`window_size=120`，输出结果长度为 11881。性能脚本输出的主要结果如下：

Listing 12: 性能分析主要结果

```

data_size: 12000
window_size: 120
result_length: 11881
brute_force_time_seconds: 0.010739
monotonic_queue_time_seconds: 0.001627
array_queue_time_seconds: 0.001540
monotonic_speedup_vs_brute_force: 6.60x
array_speedup_vs_brute_force: 6.97x
array_speed_ratio_vs_monotonic_queue: 1.06x

```

同一次运行中，`cProfile` 按累计耗时排序输出了三种算法的主要调用情况。暴力遍历版的 `profile` 结果如下：

Listing 13: 暴力遍历版 `cProfile` 主要结果

```

Brute-force cProfile top calls:
47770 function calls in 0.014 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    1  0.003    0.003   0.014   0.014 sliding_window_max.py:54(sliding_window_max_brute_force)
11881  0.008    0.000   0.008   0.000 {built-in method builtins.max}
    1  0.001    0.001   0.002   0.002 sliding_window_max.py:19(validate_window_input)
24003  0.001    0.000   0.001   0.000 {built-in method builtins.isinstance}
11881  0.000    0.000   0.000   0.000 {method 'append' of 'list' objects}
    1  0.000    0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}

```

```
2 0.000 0.000 0.000 0.000 {built-in method builtins.len}
```

单调队列版的 profile 结果如下：

Listing 14: 单调队列版 cProfile 主要结果

```
Monotonic-queue cProfile top calls:
59886 function calls in 0.007 seconds

Ordered by: cumulative time
List reduced from 9 to 8 due to restriction <8>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1  0.004    0.004    0.007    0.007 sliding_window_max.py:78(sliding_window_max_monotonic_queue)
      1  0.001    0.001    0.002    0.002 sliding_window_max.py:19(validate_window_input)
24003  0.001    0.000    0.001    0.000 {built-in method builtins.isinstance}
11881  0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
11893  0.000    0.000    0.000    0.000 {method 'pop' of 'collections.deque' objects}
12000  0.000    0.000    0.000    0.000 {method 'append' of 'collections.deque' objects}
     105  0.000    0.000    0.000    0.000 {method 'popleft' of 'collections.deque' objects}
      1  0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

数组模拟队列版的 profile 结果如下：

Listing 15: 数组模拟队列版 cProfile 主要结果

```
Array-queue cProfile top calls:
35889 function calls in 0.006 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1  0.003    0.003    0.006    0.006 sliding_window_max.py:116(sliding_window_max_array_queue)
      1  0.001    0.001    0.002    0.002 sliding_window_max.py:19(validate_window_input)
24003  0.001    0.000    0.001    0.000 {built-in method builtins.isinstance}
11881  0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
      1  0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      2  0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

本次运行中，暴力版耗时约 0.010739 秒，单调队列版耗时约 0.001627 秒，数组模拟队列版耗时约 0.001540 秒。其中，单调队列版相对暴力版约为 6.60x 加速，数组模拟队列版相对暴力版约为 6.97x 加速；数组模拟队列版相对单调队列版的速度比约为 1.06x。该结果与复杂度分析基本一致：暴力版需要对每个窗口重复调用 `max`，而单调队列相关实现只维护可能成为最大值的候选下标，因此避免了重复扫描窗口。

从 profile 结果看，暴力遍历版中 `builtins.max` 被调用 11881 次，正好对应窗口数量。该函数累计耗时约 0.008 秒，是暴力版的主要瓶颈。单调队列版虽然总函数调用次数为 59886 次，高于暴力版的 47770 次，但这些调用主要分布在 `deque.append`、`deque.pop` 和 `deque.popleft` 等常数时间操作，因此整体性能仍然明显优于暴力版。数组模拟队列版继续保留单调队列思想，但用列表和首尾指针替代部分 `deque` 方法调用，函数调用次数降为 35889 次，profile 中显示的累计时间也略低。

由此可见，主要优化来自算法层面，即避免每个窗口都重新扫描 k 个元素；数组模拟队列版的进一步优化则属于实现层面的优化，主要减少队列维护过程中的方法调用开销。数组模拟队列版相对 `deque` 版的速度比约为 1.06x，说明它在本次实验中有小幅提升，但这种提升主要是常数开销上的差异，不应理解为复杂度层面的提升。

需要说明的是，本次计时只是在固定数据规模和固定运行环境下得到的结果，运行时间可能会受到机器负载、Python 解释器状态等因素影响。因此，我们在这里不把具体秒数作为绝对结论，而是主要关注三种实现之间的相对差异。

从本次实验结果来看，三种实现的输出保持一致，说明优化没有改变算法结果。在性能方面，单调队列版和数组模拟队列版都明显快于暴力遍历版，说明避免重复扫描窗口是主要优化来源。数组模拟队列版相对 `deque` 版略快，但提升幅度较小，更多体现为实现细节上的常数优化，而不是算法复杂度上的进一步下降。

8 f) 单元测试

8.1 测试目标与被测单元

单元测试的目标是在较小粒度上验证程序模块是否满足预期行为。本实验的被测单元包括输入校验函数、三种滑动窗口最大值算法、示例运行入口和主程序入口。测试并不只是去检查最终输出，还包括检查参数类型、异常分支、循环路径、局部数据结构维护和输出行为。

表 5: 被测单元与测试目标

被测单元	测试目标
<code>validate_window_input</code>	验证输入类型、空数组、窗口边界、元素类型和布尔值排除规则
暴力遍历算法函数	验证暴力遍历基准实现的窗口最大值结果
单调队列算法函数	验证单调队列维护、队尾弹出、队首过期移除和结果追加
数组模拟队列算法函数	验证数组模拟队列的头尾指针变化和最大值候选维护
<code>run_demo、main</code>	验证示例输出和命令行入口返回状态

测试用例的期望结果以题目规定和算法定义为依据。对于合法输入，三种算法应返回完全相同的最大值数组；对于非法输入，校验函数应在核心计算开始前抛出明确异常。本实验最终统计测试通过率、语句覆盖率和分支覆盖率，并结合黑盒测试、白盒测试、Mock 测试和异常路径测试进行综合分析。

8.2 测试驱动与公共断言工具

在具体设计黑盒测试和白盒测试前，先编写统一的测试基类。该基类承担测试驱动作用：它集中准备三种算法函数，并提供复用断言方法。这样后续每一种测试方法只需要关注输入、期望输出和异常类型，不需要重复编写三种算法的调用过程。

Listing 16: 测试基类完整代码

```

1 class SlidingWindowTestCase(unittest.TestCase):
2     """提供三种算法共用的断言工具。"""
3
4     def setUp(self) -> None:
5         """准备每个测试用例需要调用的算法函数。"""
6         self.algorithms = (
```

```

7         ("brute_force", sliding_window_max_brute_force),
8         ("monotonic_queue", sliding_window_max_monotonic_queue),
9         ("array_queue", sliding_window_max_array_queue),
10    )
11
12    def tearDown(self) -> None:
13        """清理每个测试用例使用的算法函数集合。"""
14        self.algorithms = ()
15
16    def assert_algorithm_results(
17        self, nums: list[int], k: int, expected: list[int]
18    ) -> None:
19        """断言三种算法都能得到预期结果。"""
20        for algorithm_name, algorithm in self.algorithms:
21            with self.subTest(algorithm=algorithm_name):
22                self.assertEqual(algorithm(nums, k), expected)
23
24    def assert_result_length(self, nums: list[int], k: int) -> None:
25        """断言三种算法输出长度均满足  $n-k+1$ 。"""
26        expected_length = len(nums) - k + 1
27        for algorithm_name, algorithm in self.algorithms:
28            with self.subTest(algorithm=algorithm_name):
29                self.assertEqual(len(algorithm(nums, k)), expected_length)

```

`setUp` 为每个测试方法准备暴力版、单调队列版和数组模拟队列版，`tearDown` 在测试结束后清理算法函数集合。`assert_algorithm_results` 将同一输入同时送入三种算法，用于检查优化版本是否保持题目语义；`assert_result_length` 用于检查输出长度是否满足 $\text{len}(\text{nums}) - k + 1$ 。后续测试类均继承该基类，因此测试代码既能保持独立用例结构，又能避免重复调用逻辑。

8.3 单元测试基本概念与质量指标

在具体开始设计测试用例之前，我们需要先明晰一些关键概念。其中，测试用例是为特定测试目标而设计的一组输入、执行条件和期望结果，一个完整测试用例通常包含测试用例值、期望结果、前缀值和后缀值。

- **测试用例值**是被送入程序的输入；
- **期望结果**是程序满足需求时应产生的输出或异常；
- **前缀值**用于使被测程序进入适合测试的稳定状态；
- **后缀值**用于查看测试结果或结束测试过程。

在本实验中的前缀值主要是测试类初始化出的算法函数集合，后缀值主要是断言返回值、断言异常和检查 Mock 捕获到的打印内容。

另外，测试通过率是指测试过程中执行通过的测试用例占总测试用例的比例，单元测试通常要求测试用例通过率达到 100%。测试覆盖率是衡量测试完整性的手段，一般主要指代码覆盖率，它用于描述代码被测试执行的比例和程度。覆盖率数据能够指出哪些语句或分支尚未被执行，从而帮助补充测

试用例；但覆盖率不能单独代表测试质量。较低覆盖率通常说明测试不足，而较高覆盖率只能说明代码被执行过，不能证明所有输入情形和逻辑错误都已经被发现。

8.4 辅助检查思路

在正式划分黑盒测试和白盒测试前，本实验还采用 Right-BICEP 作为辅助检查清单。该清单用于提醒测试设计不要只检查“正常结果是否正确”，还要关注边界条件、反向关系、交叉检查、错误条件和性能风险。它不作为黑盒测试的主分类，而是作为补充视角帮助发现遗漏。

在辅助检查阶段，本实验选取典型输入组合、窗口移动行为和简单变形关系作为补充观察点。典型输入组合用于检查负数、小窗口、较大窗口和窗口连续右移等场景；变形关系则用于检查输入整体平移或按正数比例缩放后，输出是否保持相应关系。测试中通过把同一输入交给三种算法实现，用相同期望结果进行交叉检查，如下代码所示：

Listing 17: 补充验证、变形关系与交叉检查测试完整代码

```

1 class TestSupplementaryAlgorithmBehavior(SlidingWindowTestCase):
2     """补充验证：典型输入组合和窗口移动行为。"""
3
4     def test_negative_values_with_small_window(self) -> None:
5         """负数数组与较小窗口应得到正确最大值。"""
6         self.assert_algorithm_results([-4, -1, -7, -2], 2, [-1, -1, -2])
7
8     def test_duplicate_values_with_large_window(self) -> None:
9         """重复元素数组与较大窗口应得到正确最大值。"""
10        self.assert_algorithm_results([3, 3, 1, 3, 2], 4, [3, 3])
11
12    def test_valid_input_enters_normal_calculation(self) -> None:
13        """所有输入条件合法时应进入正常计算。"""
14        self.assert_algorithm_results([8, 3, 5], 2, [8, 5])
15
16    def test_oversized_window_raises_value_error(self) -> None:
17        """窗口越界时应产生 ValueError。"""
18        with self.assertRaises(ValueError):
19            validate_window_input([1, 2, 3], 5)
20
21    def test_window_moves_across_array(self) -> None:
22        """窗口连续右移时最大值应保持正确。"""
23        self.assert_algorithm_results([1, 4, 2, 6], 2, [4, 4, 6])
24
25
26 class TestSupplementaryMetamorphicRelation(SlidingWindowTestCase):
27     """补充验证：变形关系和交叉检查。"""
28
29     def test_translation_relation_with_added_constant(self) -> None:
30         """所有输入同时加常数时，输出也应加同一常数。"""
31         self.assert_algorithm_results([11, 13, 11, 7, 15], 3, [13, 13, 15])
32
33     def test_positive_scale_relation(self) -> None:
34         """所有输入乘以正数时，输出最大值应按比例放大。"""

```


35

```
self.assert_algorithm_results([6, -3, 12, 0], 2, [6, 12, 12])
```

这段测试代码没有引入新的被测接口，而是复用测试基类中的 `assert_algorithm_results`。因此，每个变形关系用例都会同时调用暴力版、单调队列版和数组模拟队列版；补充行为用例也能复用同一套断言工具，检查三种实现是否在典型窗口移动场景下保持一致。若某个优化版本破坏了题目语义，交叉检查能够直接暴露差异。

8.5 单元测试内容划分

单元测试内容可从模块接口、局部数据结构、边界条件、独立路径和出错处理五个方面展开。本实验按这五类内容组织测试点，并在后续黑盒测试和白盒测试中分别设计对应输入。

表 6: 单元测试内容与本实验对应关系

测试内容	概念说明	本实验对应测试点
模块接口	检查参数表、返回值、调用关系和外部接口约定	校验 <code>nums</code> 、 <code>k</code> ，确认算法返回 <code>list[int]</code>
局部数据结构	检查模块内部数据结构、初始化和状态变化	检查结果列表、 <code>deque</code> 候选下标和数组模拟队列指针
边界条件	检查数据流或控制流位于边界附近时是否出错	覆盖 <code>k=1</code> 、 <code>k=len(nums)</code> 、 <code>k=0</code> 和越界窗口
独立路径	检查主要执行路径、判定路径和循环路径	覆盖校验成功路径、异常路径、队首过期路径、队尾弹出路径
出错处理	检查错误条件是否被识别，错误处理是否正确	覆盖非列表、空数组、非法 <code>k</code> 、非整数元素和布尔值元素

8.6 静态测试与动态测试

静态测试是指在不运行程序的情况下，通过人工分析、代码审查或工具检查来发现问题。静态测试主要关注代码文本本身，包括命名、缩进、注释、导入关系、接口一致性和潜在错误。本实验使用 `pylint` 工具对三个 Python 文件进行静态检查，并结合人工模块检查确认测试代码与算法代码之间的接口关系。

动态测试是通过运行程序来检查实际行为。本实验的动态测试包括 `unittest` 测试、Mock 输出验证、覆盖率统计和示例运行结果检查。动态测试主要关注函数返回值、异常类型、打印内容、循环路径和分支执行情况。与静态测试能够发现风格和潜在结构问题的作用相比，动态测试能够验证程序在具体输入下的真实行为，总体来说两者相互补充。

8.7 黑盒测试方法与用例设计

黑盒测试又称功能测试，它将测试对象看成黑盒，不考虑程序内部逻辑结构，只依据开发者指明的需求规格说明设计输入，并根据输出结果判断功能是否符合说明。本实验的黑盒测试以滑动窗口最大值问题定义为依据：输入应为非空整数数组 `nums` 和正整数窗口大小 `k`，且 `k` 不能大于数组长度；输出应为每次窗口滑动后的最大值数组。这部分测试代码中主要按照四种黑盒测试方法进行组织，分别是等价类划分、边界值分析、健壮性测试和错误推测法，这四种方法层层递进，互为补充。

8.7.1 等价类划分

等价类划分是将输入域划分成尽可能少的若干子域，并要求各子域两两互不相交。有效等价类由合理输入数据构成，用于检验程序是否实现规定功能；无效等价类由不合理输入数据构成，用于检查程序的鲁棒性。对于同一个等价类，通常选取一个具有代表性的输入作为测试用例。

根据题目规格，本实验将输入划分为表 7 所示的等价类。合法整数数组和合法窗口大小构成有效等价类；非列表、空列表、非整数元素、布尔值元素、非整数 k 、非正 k 和超长窗口构成无效等价类。

表 7: 等价类划分

输入对象	有效等价类	无效等价类
nums	非空整数列表，如题目样例、负数数组、零值数组、重复值数组	非列表、空列表、含非整数元素、含布尔值元素
k	整数且满足 $1 \leq k \leq \text{len}(\text{nums})$	非整数、布尔值、 $k \leq 0$ 、 $k > \text{len}(\text{nums})$
输出结果	每个滑动窗口最大值组成的列表	非法输入时不进入算法计算，而是抛出明确异常

等价类测试中，题目样例、全负数数组、含零数组、全相等数组、最小合法输入和普通合法输入覆盖有效等价类；元组、None、空列表、字符串元素、布尔值元素、浮点数 k 、字符串 k 、布尔值 k 、非正 k 和超长窗口覆盖无效等价类。合法等价类由三种算法共同计算并与期望结果比较；无效等价类调用输入校验函数并断言异常类型。

本小节根据等价类划分设计的主要测试用例如下：

表 8: 等价类划分测试用例

等价类	代表输入	期望结果
普通有效数组	[1, 3, -1, -3, 5, 3, 6, 7], $k=3$	[3, 3, 5, 5, 6, 7]
全负数有效数组	[-8, -3, -5, -2, -9], $k=2$	[-3, -3, -2, -2]
含零有效数组	[0, 0, -1, 0], $k=2$	[0, 0, 0]
全相等有效数组	[7, 7, 7, 7], $k=2$	[7, 7, 7]
最小合法输入	[1], $k=1$	校验通过
非列表输入	nums=(1, 2, 3)	TypeError
空列表输入	nums=[]	ValueError
非整数元素	nums=[1, "2", 3]	TypeError
布尔值元素	nums=[1, True, 3]	TypeError
非整数窗口	$k=2.5$ 、 $k="2"$	TypeError
非正窗口	$k=0$	ValueError
超长窗口	$k=4$, nums=[1, 2, 3]	ValueError

Listing 18: 等价类划分测试完整代码

```

1 class TestBlackBoxEquivalenceClass(SlidingWindowTestCase):
2     """黑盒测试：等价类划分。"""
3
4     def test_sample_case(self) -> None:
5         """题目样例代表普通有效等价类。"""
6         nums = [1, 3, -1, -3, 5, 3, 6, 7]
7         self.assert_algorithm_results(nums, 3, [3, 3, 5, 5, 6, 7])

```



```

8
9 def test_negative_numbers(self) -> None:
10     """全负数数组代表有效整数数组等价类。"""
11     self.assert_algorithm_results(
12         [-8, -3, -5, -2, -9], 2, [-3, -3, -2, -2]
13     )
14
15 def test_zero_values(self) -> None:
16     """包含零值的数组仍属于有效整数数组等价类。"""
17     self.assert_algorithm_results([0, 0, -1, 0], 2, [0, 0, 0])
18
19 def test_all_equal_values(self) -> None:
20     """全部相等的整数数组仍属于有效等价类。"""
21     self.assert_algorithm_results([7, 7, 7, 7], 2, [7, 7, 7])
22
23 def test_validation_accepts_minimum_valid_input(self) -> None:
24     """最小合法输入应通过输入校验。"""
25     self.assertIsNone(validate_window_input([1], 1))
26
27 def test_validation_accepts_nominal_input(self) -> None:
28     """普通合法输入应通过输入校验。"""
29     self.assertIsNone(validate_window_input([1, -2, 3], 2))
30
31 def test_nums_must_be_list_tuple(self) -> None:
32     """nums 为元组时属于无效等价类。"""
33     with self.assertRaises(TypeError):
34         validate_window_input((1, 2, 3), 2) # type: ignore[arg-type]
35
36 def test_nums_must_be_list_none(self) -> None:
37     """nums 为 None 时属于无效等价类。"""
38     with self.assertRaises(TypeError):
39         validate_window_input(None, 2) # type: ignore[arg-type]
40
41 def test_nums_items_must_be_integer_string(self) -> None:
42     """字符串元素属于无效等价类。"""
43     with self.assertRaises(TypeError):
44         validate_window_input([1, "2", 3], 2) # type: ignore[list-item]
45
46 def test_k_must_be_integer_float(self) -> None:
47     """浮点数 k 属于无效等价类。"""
48     with self.assertRaises(TypeError):
49         validate_window_input([1, 2, 3], 2.5) # type: ignore[arg-type]
50
51 def test_k_must_be_integer_string(self) -> None:
52     """字符串 k 属于无效等价类。"""
53     with self.assertRaises(TypeError):
54         validate_window_input([1, 2, 3], "2") # type: ignore[arg-type]
55
56 def test_nums_cannot_be_empty_invalid_equivalence(self) -> None:

```

```

57     """空列表属于无效等价类。"""
58     with self.assertRaises(ValueError):
59         validate__window__input([], 1)
60
61     def test_bool_item_invalid_equivalence(self) -> None:
62         """布尔值元素属于无效等价类。"""
63         with self.assertRaises(TypeError):
64             validate__window__input([1, True, 3], 2)
65
66     def test_bool_k_invalid_equivalence(self) -> None:
67         """布尔值 k 属于无效等价类。"""
68         with self.assertRaises(TypeError):
69             validate__window__input([1, 2, 3], True)
70
71     def test_non_positive_k_invalid_equivalence(self) -> None:
72         """非正整数 k 属于无效等价类。"""
73         with self.assertRaises(ValueError):
74             validate__window__input([1, 2, 3], 0)
75
76     def test_oversized_k_invalid_equivalence(self) -> None:
77         """大于数组长度的 k 属于无效等价类。"""
78         with self.assertRaises(ValueError):
79             validate__window__input([1, 2, 3], 4)

```

这段代码体现了等价类划分的执行方式：有效等价类进入三种算法函数并比较输出，无效等价类只调用公共校验函数并断言异常。这样既能验证功能结果，也能确认算法主体不会接收不符合题目定义的输入。

8.7.2 边界值分析

边界值分析是在输入或输出等价类的边界附近选择测试数据的方法，通常作为等价类划分的补充。在软件开发实践中，故障往往容易出现在输入定义域或输出值域的边界附近，因此边界值分析通常选取正好等于、刚刚大于或刚刚小于边界的值，而不是只选取等价类内部的任意典型值。

本实验中，窗口大小 k 的合法范围为 $1 \leq k \leq \text{len}(\text{nums})$ 。边界值测试遵循**单缺陷思想**：一次主要改变一个变量到边界或边界附近，其他变量保持正常取值。这样可以在测试失败时更容易判断问题来自哪个边界。窗口大小的正常值记为 nom ，边界附近取 min 、 min+ 、 max- 和 max 。此外，单元素数组、输出最大值位于窗口左边界、输出最大值位于窗口右边界、结果长度 $\text{len}(\text{nums}) - k + 1$ 等用例用于检查输入边界和输出边界是否一致。

本小节根据边界值分析设计的测试用例如下。除被检查的边界变量外，其余输入均保持为合法整数数组。

表 9: 边界值分析对应测试用例

边界位置	代表输入	预期结果
min	[2,-1,5,0], k=1	输出原数组 [2,-1,5,0]
min+	[4,1,3,2], k=2	[4,3,3]
nom	[6,1,5,2,4,3], k=3	[6,5,5,4]
max-	[1,9,2,8,3], k=4	[9,9]
max	[4,1,8,2], k=4	[8]
输出边界	最大值位于窗口左边界或右边界	窗口滑动后最大值及时保留或更新

Listing 19: 边界值分析测试完整代码

```

1 class TestBlackBoxBoundaryValueAnalysis(SlidingWindowTestCase):
2     """黑盒测试：边界值分析。"""
3
4     def test_single_element_array(self) -> None:
5         """数组长度和窗口长度同时取最小合法值。"""
6         self.assert_algorithm_results([42], 1, [42])
7
8     def test_window_size_is_one(self) -> None:
9         """窗口长度取下边界 min。"""
10        nums = [2, -1, 5, 0]
11        self.assert_algorithm_results(nums, 1, nums)
12
13    def test_window_size_is_two(self) -> None:
14        """窗口长度取 min+。"""
15        self.assert_algorithm_results([4, 1, 3, 2], 2, [4, 3, 3])
16
17    def test_window_size_is_nominal_value(self) -> None:
18        """窗口长度取普通中间值 nom。"""
19        self.assert_algorithm_results([6, 1, 5, 2, 4, 3], 3, [6, 5, 5, 4])
20
21    def test_window_size_is_max_minus(self) -> None:
22        """窗口长度取 max-。"""
23        self.assert_algorithm_results([1, 9, 2, 8, 3], 4, [9, 9])
24
25    def test_window_size_equals_array_length(self) -> None:
26        """窗口长度取上边界 max。"""
27        self.assert_algorithm_results([4, 1, 8, 2], 4, [8])
28
29    def test_maximum_at_window_left_boundary(self) -> None:
30        """输出值来自窗口左边界时应保持正确。"""
31        self.assert_algorithm_results([9, 1, 1, 1], 2, [9, 1, 1])
32
33    def test_maximum_at_window_right_boundary(self) -> None:
34        """输出值来自窗口右边界时应及时更新。"""
35        self.assert_algorithm_results([1, 1, 1, 9], 2, [1, 1, 9])
36
37    def test_result_length_follows_formula(self) -> None:

```

```

38     """输出数组长度边界应满足 len(nums)-k+1。"""
39     self.assert_result_length([3, 1, 4, 1, 5, 9], 4)

```

这段测试代码承接等价类测试中的公共断言工具。边界用例仍然调用三种算法，而不是只检查某一个版本，从而保证边界行为在不同实现中保持一致。

8.7.3 健壮性测试

健壮性测试则是对边界值分析的进一步扩充。边界值分析通常考虑 min、min+、nom、max- 和 max，健壮性测试还增加 min- 和 max+，用于检查超过极限值时系统是否能稳定处理错误输入。

对于本实验的 k，min- 对应 k=0 和负数 k，max+ 对应 k=len(nums)+1。数组长度的越界输入对应空数组；类型边界输入包括布尔值 k、浮点数元素、None 元素和嵌套列表元素。测试结果要求：合法边界返回正确最大值数组；越界输入或非法类型输入抛出 ValueError 或 TypeError，而不是进入核心算法后产生不可控错误。

本小节根据健壮性测试设计的测试用例如下。与边界值分析相比，这些用例增加了边界外侧输入，重点检查错误处理是否稳定。

表 10: 健壮性测试用例

健壮性位置	代表输入	预期结果
min-	k=0、k=-1	抛出 ValueError
max+	nums=[1,2,3], k=4	抛出 ValueError
数组长度越界	nums=[]	抛出 ValueError
特殊类型边界	k=True	抛出 TypeError
元素类型越界	[1,2.0,3]、[1,None,3]	抛出 TypeError
嵌套元素	[1,[2],3]	抛出 TypeError

Listing 20: 健壮性测试完整代码

```

1 class TestBlackBoxRobustnessTesting(SlidingWindowTestCase):
2     """黑盒测试：健壮性测试。"""
3
4     def test_nums_cannot_be_empty(self) -> None:
5         """空数组是数组长度的越界输入。"""
6         with self.assertRaises(ValueError):
7             validate_window_input([], 1)
8
9     def test_k_must_be_positive_zero(self) -> None:
10        """k=0 对应窗口大小 min-。"""
11        with self.assertRaises(ValueError):
12            validate_window_input([1, 2, 3], 0)
13
14    def test_k_must_be_positive_negative(self) -> None:
15        """负数 k 是小于最小边界的非法输入。"""
16        with self.assertRaises(ValueError):
17            validate_window_input([1, 2, 3], -1)
18
19    def test_k_cannot_exceed_array_length(self) -> None:

```

```

20     """k=len(nums)+1 对应窗口大小 max+。"""
21     with self.assertRaises(ValueError):
22         validate_window_input([1, 2, 3], 4)
23
24     def test_k_must_not_be_bool(self) -> None:
25         """布尔值 k 是类型边界上的特殊非法输入。"""
26         with self.assertRaises(TypeError):
27             validate_window_input([1, 2, 3], True)
28
29     def test_nums_items_must_be_integer_float(self) -> None:
30         """浮点数元素是元素类型边界上的非法输入。"""
31         with self.assertRaises(TypeError):
32             validate_window_input([1, 2.0, 3], 2) # type: ignore[list-item]
33
34     def test_nums_items_must_be_integer_none(self) -> None:
35         """None 元素是元素类型边界上的非法输入。"""
36         with self.assertRaises(TypeError):
37             validate_window_input([1, None, 3], 2) # type: ignore[list-item]
38
39     def test_nested_list_item_is_rejected(self) -> None:
40         """嵌套列表元素是元素类型边界上的非法输入。"""
41         with self.assertRaises(TypeError):
42             validate_window_input([1, [2], 3], 2) # type: ignore[list-item]

```

这段代码与输入校验函数直接衔接，健壮性测试一般不追求进入算法主体，而是验证非法输入能否在入口处被识别，并且异常类型与错误原因保持一致。

8.7.4 错误推测法

错误推测法是根据经验或直觉推测程序可能出现错误的位置，并有针对性地设计测试用例的方法。滑动窗口最大值问题中容易出现的错误包括：重复最大值导致队列维护错误、严格递增数组导致队尾弹出错误、严格递减数组导致队首过期处理错误、布尔值被 Python 误认为整数等。

错误推测法对应测试类 `TestBlackBoxErrorGuessing`。重复最大值数组用于检查单调队列在相等元素情况下的弹出策略；递增数组用于检查新元素连续替换候选值时结果是否正确；递减数组用于检查旧最大值滑出窗口后是否及时失效；布尔值元素用于检查 `bool` 作为 `int` 子类带来的类型误判。

本小节根据错误推测法设计的测试用例如下：

表 11: 错误推测法测试用例

可能错误	代表输入	期望结果
重复最大值维护错误	<code>[5,5,5,2,5]</code> , <code>k=3</code>	<code>[5,5,5]</code>
递增数组队尾弹出错误	<code>[1,2,3,4,5]</code> , <code>k=3</code>	<code>[3,4,5]</code>
递减数组队首过期错误	<code>[5,4,3,2,1]</code> , <code>k=3</code>	<code>[5,4,3]</code>
布尔值被误判为整数	<code>[1,True,3]</code> , <code>k=2</code>	抛出 <code>TypeError</code>

Listing 21: 错误推测法测试完整代码

```

1 class TestBlackBoxErrorGuessing(SlidingWindowTestCase):

```

```

2 """黑盒测试：错误推测法。"""
3
4 def test_duplicate_values(self) -> None:
5     """重复最大值容易导致候选下标维护错误。"""
6     self.assert_algorithm_results([5, 5, 5, 2, 5], 3, [5, 5, 5])
7
8 def test_strictly_increasing_numbers(self) -> None:
9     """递增数组容易暴露队尾弹出错误。"""
10    self.assert_algorithm_results([1, 2, 3, 4, 5], 3, [3, 4, 5])
11
12 def test_strictly_decreasing_numbers(self) -> None:
13    """递减数组容易暴露队首过期处理错误。"""
14    self.assert_algorithm_results([5, 4, 3, 2, 1], 3, [5, 4, 3])
15
16 def test_bool_is_not_accepted_as_integer_item(self) -> None:
17    """布尔值元素容易被 Python 误认为整数。"""
18    with self.assertRaises(TypeError):
19        validate_window_input([1, True, 3], 2)

```

这些用例来自滑动窗口最大值实现中最容易出错的具体位置。它们一方面补充了等价类和边界值测试未必覆盖的特殊数据形态，另一方面也为后续白盒测试中的队列维护路径提供了输入依据。

8.8 白盒测试方法与用例设计

白盒测试又称结构测试，它将测试对象看成透明盒子，依据程序内部逻辑结构和控制流信息设计测试用例。与只关注输入输出关系的黑盒测试不同，白盒测试要求分析程序中的判断、循环、异常分支和主要路径。本实验的白盒算法测试只针对单调队列算法展开；由于单调队列函数在进入核心循环前会调用输入校验函数，因此在白盒分析时我们会同时列出输入校验的异常路径。这样既覆盖算法主体，也覆盖算法入口处可能提前退出的控制流。

8.8.1 基于控制流的测试过程

基于控制流的测试首先需要确定程序单元和路径选择标准，然后画出程序流程图，再将流程图抽象为控制流图。控制流图中的节点表示语句或基本块，边表示控制流方向。测试人员需要根据覆盖标准选择路径，并为所选路径生成测试输入数据；若路径不可行，则重新选择路径。本实验按照该流程先确定单调队列算法为白盒算法测试对象，并把输入校验作为前置控制流纳入路径分析，再依据覆盖标准设计具体输入。

在本实验中，路径选择标准包括：输入校验的成功路径和所有异常路径必须覆盖；算法循环的主要路径必须覆盖；单调队列中的队首过期移除、队尾弹出和结果追加路径必须覆盖；示例入口和主入口也应至少执行一次。

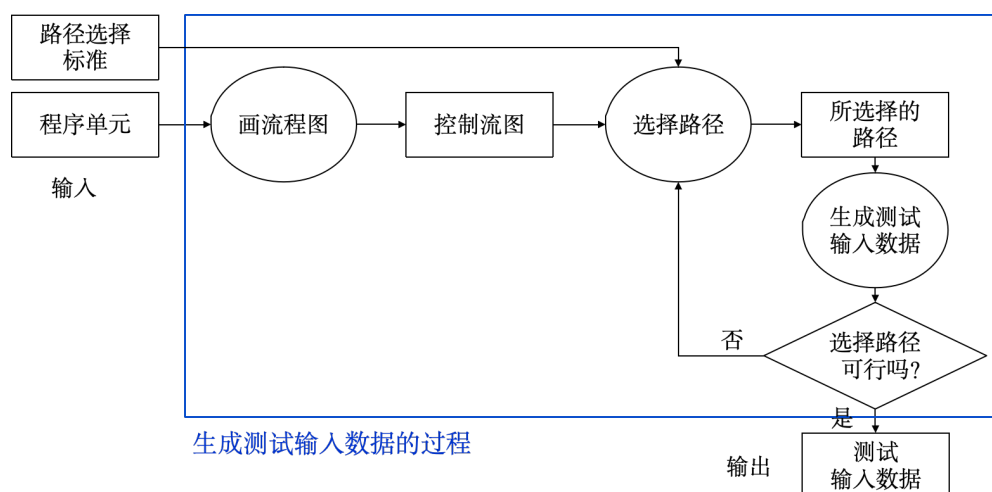


图 1: 基于控制流的测试过程

8.8.2 流程图与控制流图

输入校验函数负责在算法运行前筛选非法输入，下面这张图则描述了该函数的流程图。该流程从输入开始，依次检查 `nums` 类型、数组是否为空、`k` 类型、窗口大小范围和数组元素类型。一旦某个条件不满足，程序立即抛出异常；只有所有检查均通过时，算法函数才进入核心计算。

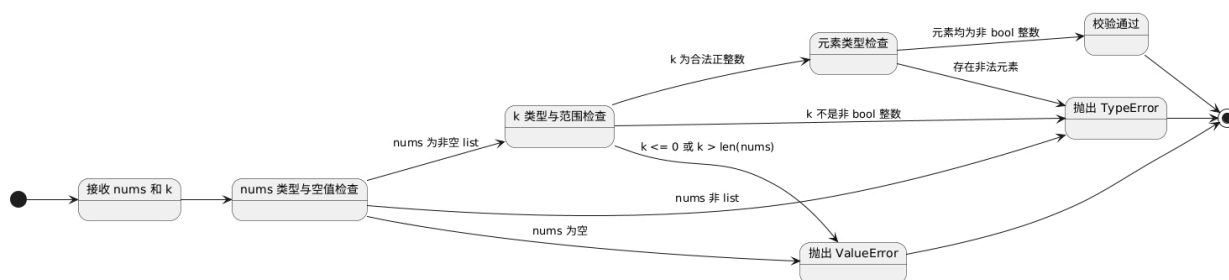


图 2: 输入校验函数流程图

根据流程图可得到控制流图。控制流图将顺序执行语句合并为基本块，更突出判断节点和异常退出路径。节点 1 至节点 7 分别对应输入类型、空数组、窗口类型、窗口正数、窗口长度、元素类型和校验成功；节点 E 表示异常退出。该图直接用于后续路径覆盖分析。

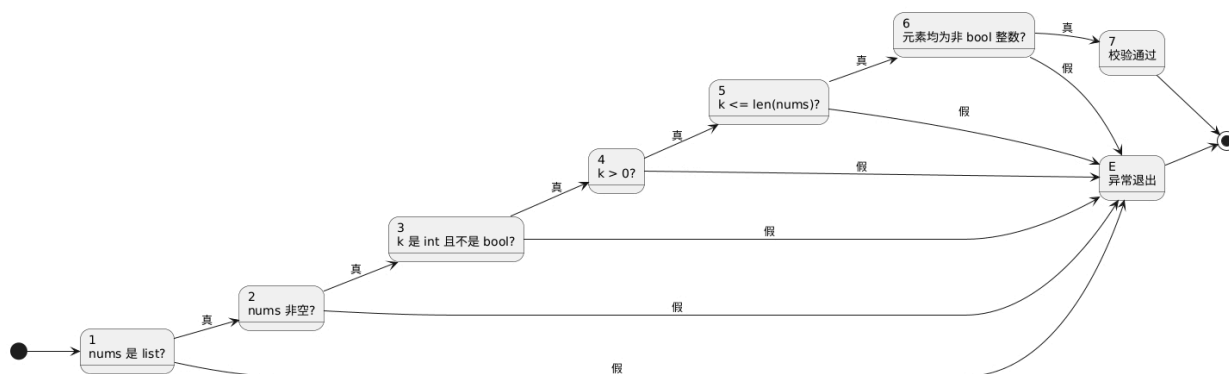


图 3: 输入校验函数控制流图

前面的流程图主要说明算法开始前的输入校验过程。为了分析单调队列算法的核心逻辑，还需要进一步展开循环内部的处理流程。算法遍历数组时，使用一个队列保存仍有可能成为窗口最大值的下标，并使这些下标对应的元素值从队首到队尾保持单调递减。

每处理一个新元素，算法先移除已经离开当前窗口的队首下标，再从队尾删除对应值小于或等于当前元素的下标，最后将当前下标加入队列。当窗口形成后，队首下标对应的元素就是当前窗口的最大值。以下流程图展示了输入校验、数组遍历、队首过期移除、队尾候选更新和窗口结果追加的完整过程。

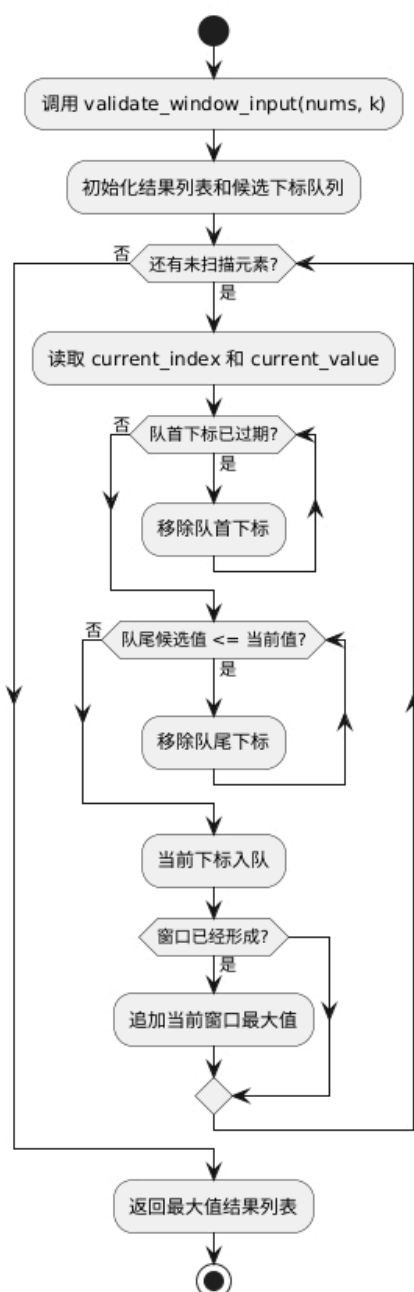


图 4: 单调队列算法流程图

根据该流程图我们可以进一步抽象出单调队列算法的控制流图。该图中的节点 A 对应输入校验，节点 B 对应初始化，节点 C 对应主循环取下一个元素，节点 D 与 E 对应队首过期判断和移除，节点

F 与 G 对应队尾候选比较和弹出，节点 H 对应当前下标入队，节点 I 与 J 对应窗口形成判断和结果追加。白盒测试需要围绕这些判断节点构造输入，使队首过期、队尾弹出、队尾保留、窗口未形成和窗口已形成等路径都能被执行。

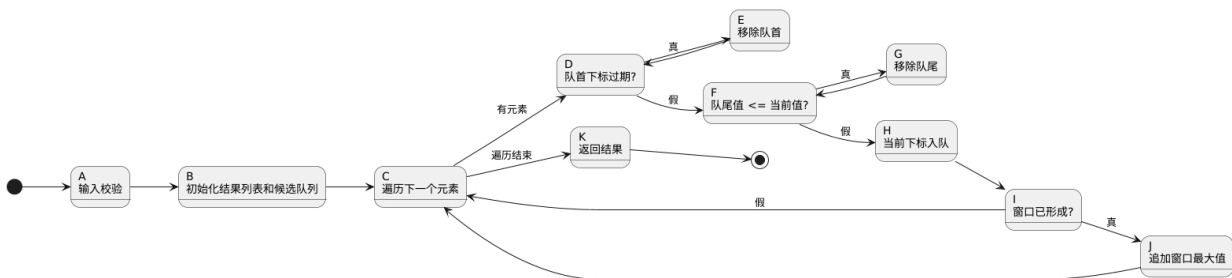


图 5: 单调队列算法控制流图

8.8.3 测试需求、覆盖标准和路径选择标准

在开始具体设计白盒测试用例之前，我们同样需要先明晰一些关键概念：

- **测试需求**是软件制品中必须由测试用例满足或覆盖的特定元素。
- **覆盖标准**是将测试需求施加到测试集上的规则。
- **测试覆盖**表示对于测试需求集合中的每条需求，至少存在一个测试用例能够满足它。
- **覆盖程度**是已满足测试需求数占全部测试需求数的比例。

另外，路径选择标准则用于决定控制流图中的哪些路径应被选入测试集。本实验的主要测试需求包括：所有公开函数接口至少被调用一次；输入校验函数中的每个异常分支至少被触发一次；单调队列算法至少在正常输入下计算一次；单调队列中的候选维护路径至少被覆盖一次；示例入口和主入口至少被执行一次。覆盖标准采用语句覆盖、判定覆盖、条件覆盖、判定条件覆盖、条件组合覆盖、路径覆盖的组合说明，并用 `coverage.py` 实际统计语句覆盖率和分支覆盖率，`coverage.py` 是 Python 常用的第三方覆盖率统计工具，通过命令行即可调用该工具。

8.8.4 六种代码覆盖标准

代码覆盖率用于描述代码被测试的比例和程度，常用覆盖标准包括语句覆盖、判定覆盖、条件覆盖、判定条件覆盖、条件组合覆盖和路径覆盖。表 12 给出了各覆盖标准的定义以及我们本实验中给出的对应的测试设计方式。

表 12: 白盒覆盖标准与本实验用例设计

覆盖标准	概念定义	本实验构造方式
语句覆盖	程序中的每个可执行语句至少被执行一次	正常样例执行算法主体，异常用例执行校验分支，入口测试执行 <code>run_demo</code> 和 <code>main</code>
判定覆盖	每个判断的真、假分支至少经历一次	分别构造合法输入和非法输入，使类型判断、空数组判断、窗口范围判断、队列状态判断均出现真假结果

覆盖标准	概念定义	本实验构造方式
条件覆盖	每个判断中每个条件的可能取值至少满足一次	对 <code>not isinstance(value, int) or isinstance(value, bool)</code> 构造普通整数、字符串元素和布尔值元素
判定条件覆盖	条件真假和判断整体真假均至少出现一次	使用合法整数元素使整体判断为假, 使用非整数和布尔值使整体判断为真
条件组合覆盖	判断中各条件可能取值组合至少执行一次	针对 <code>nums</code> 类型、空数组、 <code>k</code> 类型、窗口范围和元素类型构造主要有效组合与无效组合
路径覆盖	覆盖程序中主要可能执行路径	覆盖校验成功路径、各异常退出路径、算法循环路径、队首过期路径、队尾弹出路径、入口输出路径

本部分根据六种覆盖标准设计的测试用例，主要集中在输入校验分支和单调队列维护分支。以下完整测试类展示了单调队列算法在判定覆盖、条件覆盖和路径覆盖中的对应写法。

需要说明的是，这六种覆盖标准并不是只通过本小节列出的白盒测试用例体现的，前文中的等价类划分、边界值分析、健壮性测试和错误推测用例，也会执行输入校验判断、异常分支以及不同条件组合。因此，这些测试与本小节的单调队列路径测试共同构成了覆盖率结果的来源。

Listing 22: 白盒覆盖标准测试完整代码

```

1 class TestWhiteBoxCoverageCriteria(SlidingWindowTestCase):
2     """白盒测试：六种代码覆盖标准。"""
3
4     def test_monotonic_queue_expires_front_candidate(self) -> None:
5         """覆盖单调队列队首过期分支。"""
6         self.assertEqual(
7             sliding_window_max_monotonic_queue([9, 1, 2, 3], 2),
8             [9, 2, 3],
9         )
10
11     def test_monotonic_queue_pops_smaller_tail(self) -> None:
12         """覆盖单调队列队尾弹出分支。"""
13         self.assertEqual(
14             sliding_window_max_monotonic_queue([1, 2, 3, 4], 2),
15             [2, 3, 4],
16         )
17
18     def test_monotonic_queue_keeps_larger_tail(self) -> None:
19         """覆盖单调队列队尾不弹出的分支。"""
20         self.assertEqual(
21             sliding_window_max_monotonic_queue([4, 3, 2, 1], 2),
22             [4, 3, 2],
23         )
24
25     def test_monotonic_queue_handles_equal_values(self) -> None:

```

```

26     """覆盖单调队列相等元素条件组合。"""
27     self.assertEqual(
28         sliding_window_max_monotonic_queue([2, 2, 2, 1], 2),
29         [2, 2, 2],
30     )

```

上述用例主要覆盖了单调队列中的几个关键分支，包括队首元素过期、队尾较小元素弹出、队尾较大元素保留以及相等元素处理等情况。结合前面的异常输入测试，可以覆盖输入校验、元素类型判断和单调队列维护过程中的主要判断分支。

需要说明的是，滑动窗口算法中包含循环结构，数组长度和窗口大小变化后，会产生大量重复的循环路径。因此，本实验中的路径覆盖主要针对核心控制流路径进行设计，而不是穷举所有可能的循环次数组合。最终通过覆盖率报告确认核心代码的语句和分支均已被实际执行。

8.8.5 基本路径测试与循环测试

基本路径测试是在控制流图基础上，通过分析控制结构的环路复杂度，导出基本可执行路径集合，再根据每一条基本路径设计测试用例的方法。按照基于控制流的测试过程，本实验在前文已经给出了程序流程图和控制流图，接着我们需要计算环路复杂度，确定基本路径集合，最后为这些路径选择具体输入数据。

输入校验函数包含 `nums` 类型、空数组、`k` 类型、`k > 0`、`k <= len(nums)` 和元素类型六个判定节点，因此其环路复杂度可按判定节点数加一得到：

$$V(G) = 6 + 1 = 7$$

这说明输入校验至少需要覆盖一条校验成功路径和六类异常退出路径，前文等价类划分、健壮性测试以及异常输入测试已经分别覆盖这些路径。

单调队列算法是白盒测试中的核心算法对象。按单调队列算法控制流图统计，入口、出口以及节点 A 至 K 共 $N = 13$ 个节点，边数为 $E = 16$ ，连通分量 $P = 1$ ，因此：

$$V(G) = E - N + 2P = 16 - 13 + 2 \times 1 = 5$$

该结果也可以由判定节点数进行校验：主循环判断、队首过期判断、队尾候选比较和窗口形成判断共 4 个判定节点，因此 $V(G) = 4 + 1 = 5$ ，环路复杂度为 5，说明单调队列算法至少需要选择 5 条线性独立路径作为基本路径测试依据。独立路径与其他路径相比，至少应包含一条新的边。本实验据此选择窗口立即形成、窗口尚未形成、队首过期、队尾弹出和队尾保留五类路径。

表 13: 单调队列基本路径与测试输入

路径	路径含义	对应测试输入
BP1	第一次循环时窗口立即形成并追加结果	[6], k=1
BP2	前几次循环窗口尚未形成，暂不追加结果	[6, 1], k=2
BP3	队首下标离开当前窗口，需要移除队首候选	[9, 1, 2, 3], k=2
BP4	当前值较大，队尾候选被弹出	[1, 2, 3, 4], k=2
BP5	当前值较小，队尾候选被保留	[4, 3, 2, 1], k=2

循环测试用于检查循环结构在边界次数和不同组合关系下是否有效。结合单调队列算法的控制流，

本实验将循环测试分为简单循环、嵌套循环、串接循环和非结构循环四类：简单循环关注主循环执行次数，嵌套循环关注主循环内部 `while` 循环的连续执行，串接循环关注两个 `while` 循环的前后依赖关系；当前程序不存在非结构循环，故在此不涉及。具体测试输入和预期结果见表 14。

表 14: 循环测试类型与测试用例

循环类型	测试关注点	对应用例	输入与预期结果
简单循环	主循环不进入算法主体	<code>test_nums_cannot_be_empty</code>	<code>nums=[]</code> ，期望 <code>ValueError</code>
简单循环	主循环执行 1 次	<code>test_algorithm_loop_executes_once</code>	<code>[6]</code> ， <code>k=1</code> ，期望 <code>[6]</code>
简单循环	主循环执行 2 次	<code>test_algorithm_loop_executes_twice</code>	<code>[6,1]</code> ， <code>k=1</code> ，期望 <code>[6,1]</code>
简单循环	主循环执行多次	<code>test_algorithm_loop_executes_many_times</code>	<code>[4,1,7,0,3]</code> ， <code>k=2</code> ，期望 <code>[4,7,7,3]</code>
嵌套循环	内层队尾弹出循环连续执行	<code>test_nested_loop_repeated_tail_pops</code>	<code>[1,3,2,4]</code> ， <code>k=3</code> ，期望 <code>[3,4]</code>
串接循环	队首过期循环后接队尾弹出循环	<code>test_concatenated_loops_expire_then_pop</code>	<code>[9,1,10]</code> ， <code>k=2</code> ，期望 <code>[9,10]</code>
非结构循环	程序不存在该类循环	不单独设计用例	源码由结构化 <code>for</code> 和 <code>while</code> 构成

本小节根据基本路径测试和循环测试设计的完整代码如下：

Listing 23: 基本路径与循环测试完整代码

```

1 class TestWhiteBoxBasicPathAndLoopTesting(SlidingWindowTestCase):
2     """白盒测试：基本路径测试和循环测试。"""
3
4     def test_basic_path_window_forms_immediately(self) -> None:
5         """基本路径：窗口在第一次循环时立即形成。"""
6         self.assertEqual(sliding_window_max_monotonic_queue([6], 1), [6])
7
8     def test_basic_path_window_waits_before_output(self) -> None:
9         """基本路径：窗口尚未形成时不追加结果。"""
10        self.assertEqual(
11            sliding_window_max_monotonic_queue([6, 1], 2),
12            [6],
13        )
14
15    def test_basic_path_expires_front_candidate(self) -> None:
16        """基本路径：队首候选下标过期并被移除。"""
17        self.assertEqual(
18            sliding_window_max_monotonic_queue([9, 1, 2, 3], 2),
19            [9, 2, 3],
20        )
21
22    def test_basic_path_pops_tail_candidate(self) -> None:
23        """基本路径：队尾候选值较小并被弹出。"""
24        self.assertEqual(
25            sliding_window_max_monotonic_queue([1, 2, 3, 4], 2),

```

```

26         [2, 3, 4],
27     )
28
29     def test_basic_path_keeps_tail_candidate(self) -> None:
30         """基本路径：队尾候选值较大并被保留。"""
31         self.assertEqual(
32             sliding_window_max_monotonic_queue([4, 3, 2, 1], 2),
33             [4, 3, 2],
34         )
35
36     def test_algorithm_loop_executes_once(self) -> None:
37         """单调队列主循环执行一次。"""
38         self.assertEqual(sliding_window_max_monotonic_queue([6], 1), [6])
39
40     def test_algorithm_loop_executes_twice(self) -> None:
41         """单调队列主循环执行两次。"""
42         self.assertEqual(
43             sliding_window_max_monotonic_queue([6, 1], 1),
44             [6, 1],
45         )
46
47     def test_algorithm_loop_executes_many_times(self) -> None:
48         """单调队列主循环执行多次。"""
49         self.assertEqual(
50             sliding_window_max_monotonic_queue([4, 1, 7, 0, 3], 2),
51             [4, 7, 7, 3],
52         )
53
54     def test_nested_loop_repeated_tail_pops(self) -> None:
55         """嵌套循环中队尾弹出循环可连续执行。"""
56         self.assertEqual(
57             sliding_window_max_monotonic_queue([1, 3, 2, 4], 3),
58             [3, 4],
59         )
60
61     def test_concatenated_loops_expire_then_pop(self) -> None:
62         """串接循环中先移除过期队首再弹出较小队尾。"""
63         self.assertEqual(
64             sliding_window_max_monotonic_queue([9, 1, 10], 2),
65             [9, 10],
66         )

```

前五个方法属于基本路径测试，对应环路复杂度得到的 5 条线性独立路径。后五个方法属于循环测试：前三个方法覆盖主循环执行一次、两次和多次，后两个方法分别覆盖嵌套循环和串接循环。非结构循环在当前程序中不存在，因此不构造相应用例。

8.9 unittest 与 Mock 测试实现

`unittest` 是 Python 标准库中的 xUnit 风格测试框架。编写测试时，测试类继承标准库测试基类，测试方法以 `test_` 开头，并通过断言方法判断实际结果是否符合预期。本实验使用 `setUp` 准备三种算法函数；使用 `assertEqual` 判断返回值，使用 `assertRaises` 判断异常；同时使用 `subTest` 分别记录三种算法在同一输入下的执行结果。

本实验按照以下步骤组织 `unittest` 测试代码：

1. 第一，导入 `unittest` 以及 Mock 所需的 `patch` 和 `call`；
2. 第二，定义继承自 `unittest.TestCase` 的测试用例类；
3. 第三，通过 `setUp` 和 `tearDown` 完成每个测试方法前后的准备与清理；
4. 第四，定义以 `test_` 开头的测试方法；
5. 第五，每个测试方法只验证一个明确方面，并使用 `assertEqual`、`assertRaises`、`assertIn`、`assert_has_calls` 和 `assert_called_once_with` 判断实际结果是否符合预期；
6. 第六，在文件末尾提供 `unittest.main(verbosity=2)` 入口；
7. 第七，通过 `python -m unittest -v` 运行测试，测试通过时输出 `ok`，测试失败时输出对应错误信息。

公共测试基类已经在测试设计开始处列出，本小节重点说明 `unittest` 框架的断言方式和 Mock 隔离测试。测试类负责调用被测函数，并用断言判断实际结果是否等于期望结果；不同测试方法按黑盒测试、白盒测试和输出隔离测试分别组织，便于从测试输出中直接看出用例来源。

Mock 测试使用虚拟对象替代真实对象，适用于真实对象难以构造、行为不可预测、执行速度慢、包含界面输出，或需要检查调用行为的场景。本实验算法函数为纯函数，不依赖数据库、网络或复杂下层模块，因此不需要桩模块。测试类和测试方法承担驱动模块作用；示例入口 `run_demo` 会调用 `print`，因此使用 `unittest.mock.patch` 替代真实打印，并检查打印内容和调用顺序是否符合预期。对于需要控制依赖对象行为的场景，Mock 对象可以通过 `return_value` 指定固定返回值，也可以通过 `side_effect` 指定返回序列或自定义行为。本实验在 `test_run_demo_uses_mocked_algorithm_results` 中使用 `return_value` 临时控制三个算法函数的返回值，再用 `assert_called_once_with` 和 `assert_has_calls` 检查示例入口对算法函数和打印函数的调用是否符合预期。

Listing 24: unittest 与 Mock 测试完整代码

```

1 class TestUnittestAndMockIsolation(SlidingWindowTestCase):
2     """unittest 框架和 Mock 隔离测试。"""
3
4     def test_run_demo_prints_required_sample_result(self) -> None:
5         """Mock 捕获示例入口打印的样例结果。"""
6         with patch("builtins.print") as mock_print:
7             run_demo()
8
9         printed_lines = [call_item.args[0] for call_item in mock_print.call_args_list]
10        self.assertIn("brute_force = [3, 3, 5, 5, 6, 7]", printed_lines)
11        self.assertIn("monotonic_queue = [3, 3, 5, 5, 6, 7]", printed_lines)
12        self.assertIn("array_queue = [3, 3, 5, 5, 6, 7]", printed_lines)

```

```

13
14 def test_run_demo_print_order_with_mock(self) -> None:
15     """Mock 断言示例入口打印调用顺序。"""
16     with patch("builtins.print") as mock_print:
17         run_demo()
18
19     mock_print.assert_has_calls(
20         [
21             call("nums = [1, 3, -1, -3, 5, 3, 6, 7]"),
22             call("k = 3"),
23             call("brute_force = [3, 3, 5, 5, 6, 7]"),
24             call("monotonic_queue = [3, 3, 5, 5, 6, 7]"),
25             call("array_queue = [3, 3, 5, 5, 6, 7]"),
26         ]
27     )
28
29 def test_run_demo_uses_mocked_algorithm_results(self) -> None:
30     """Mock 控制算法返回值并验证示例入口调用参数。"""
31     with (
32         patch(
33             "sliding_window_max.sliding_window_max_brute_force",
34             return_value=[10],
35         ) as mock_brute_force,
36         patch(
37             "sliding_window_max.sliding_window_max_monotonic_queue",
38             return_value=[20],
39         ) as mock_monotonic_queue,
40         patch(
41             "sliding_window_max.sliding_window_max_array_queue",
42             return_value=[30],
43         ) as mock_array_queue,
44         patch("builtins.print") as mock_print,
45     ):
46         run_demo()
47
48     expected_nums = [1, 3, -1, -3, 5, 3, 6, 7]
49     mock_brute_force.assert_called_once_with(expected_nums, 3)
50     mock_monotonic_queue.assert_called_once_with(expected_nums, 3)
51     mock_array_queue.assert_called_once_with(expected_nums, 3)
52     mock_print.assert_has_calls(
53         [
54             call("brute_force = [10]"),
55             call("monotonic_queue = [20]"),
56             call("array_queue = [30]"),
57         ]
58     )
59
60 def test_main_returns_success_status(self) -> None:
61     """主入口应返回正常结束状态码。"""

```



```

62     with patch("builtins.print"):
63         self.assertEqual(run_sample_main([], 0)
64
65     def test_main_accepts_unused_arguments(self) -> None:
66         """主入口接收额外参数时仍应返回正常状态码。"""
67         with patch("builtins.print"):
68             self.assertEqual(run_sample_main(["unused"]), 0)

```

异常路径测试覆盖输入校验控制流图中的异常出口，每个异常用例只改变一个关键输入条件，便于判断失败原因。Mock 测试则隔离了打印动作，使示例入口的输出可以被断言，从而覆盖普通算法测试无法直接检查的输出行为；对算法函数的 Mock 则进一步验证了示例入口与下层算法函数之间的调用关系。

8.10 测试用例总表

表 15 至表 17 汇总全部 63 个单元测试用例，包括黑盒测试、白盒测试与 Mock 测试。

表 15: 单元测试用例表（一）：等价类划分与边界值分析

序号	测试方法	unittest 方法	输入或操作、预期结果
1	等价类划分	test_sample_case	题目样例，期望 [3,3,5,5,6,7]
2	等价类划分	test_negative_numbers	全负数数组，期望 [-3,-3,-2,-2]
3	等价类划分	test_zero_values	含零数组，期望 [0,0,0]
4	等价类划分	test_all_equal_values	全相等数组，期望 [7,7,7]
5	等价类划分	test_validation_accepts_minimum_valid_input	[1], k=1, 期望校验通过
6	等价类划分	test_validation_accepts_nominal_input	[1,-2,3], k=2, 期望校验通过
7	等价类划分	test_nums_must_be_list_tuple	nums=(1,2,3), 期望 TypeError
8	等价类划分	test_nums_must_be_list_none	nums=None, 期望 TypeError
9	等价类划分	test_nums_items_must_be_integer_string	nums=[1,"2",3], 期望 TypeError
10	等价类划分	test_k_must_be_integer_float	k=2.5, 期望 TypeError
11	等价类划分	test_k_must_be_integer_string	k="2", 期望 TypeError
12	等价类划分	test_nums_cannot_be_empty_invalid_equivalence	空数组，期望 ValueError
13	等价类划分	test_bool_item_invalid_equivalence	布尔值元素，期望 TypeError
14	等价类划分	test_bool_k_invalid_equivalence	k=True, 期望 TypeError
15	等价类划分	test_non_positive_k_invalid_equivalence	k=0, 期望 ValueError
16	等价类划分	test_oversized_k_invalid_equivalence	k>len(nums), 期望 ValueError
17	边界值分析	test_single_element_array	[42], k=1, 期望 [42]
18	边界值分析	test_window_size_is_one	k=min=1, 期望输出原数组
19	边界值分析	test_window_size_is_two	k=min+=2, 期望 [4,3,3]
20	边界值分析	test_window_size_is_nominal_value	k=nom=3, 期望 [6,5,5,4]
21	边界值分析	test_window_size_is_max_minus	k=max-, 期望 [9,9]
22	边界值分析	test_window_size_equals_array_length	k=max=len(nums), 期望 [8]
23	边界值分析	test_maximum_at_window_left_boundary	最大值在左边界，期望 [9,1,1]
24	边界值分析	test_maximum_at_window_right_boundary	最大值在右边界，期望 [1,1,9]
25	边界值分析	test_result_length_follows_formula	输出长度为 len(nums) - k + 1

表 16: 单元测试用例表 (二): 健壮性、错误推测与补充验证

序号	测试方法	unittest 方法	输入或操作、预期结果
26	健壮性测试	test_nums_cannot_be_empty	空数组, 期望 ValueError
27	健壮性测试	test_k_must_be_positive_zero	k=0, 期望 ValueError
28	健壮性测试	test_k_must_be_positive_negative	k=-1, 期望 ValueError
29	健壮性测试	test_k_cannot_exceed_array_length	k=len(nums)+1, 期望 ValueError
30	健壮性测试	test_k_must_not_be_bool	k=True, 期望 TypeError
31	健壮性测试	test_nums_items_must_be_integer_float	浮点数元素, 期望 TypeError
32	健壮性测试	test_nums_items_must_be_integer_none	None 元素, 期望 TypeError
33	健壮性测试	test_nested_list_item_is_rejected	嵌套列表元素, 期望 TypeError
34	错误推测法	test_duplicate_values	重复最大值数组, 期望 [5,5,5]
35	错误推测法	test_strictly_increasing_numbers	严格递增数组, 期望 [3,4,5]
36	错误推测法	test_strictly_decreasing_numbers	严格递减数组, 期望 [5,4,3]
37	错误推测法	test_bool_is_not_accepted_as_integer_item	布尔值元素, 期望 TypeError
38	补充验证	test_negative_values_with_small_window	负数数组与小窗口, 期望 [-1,-1,-2]
39	补充验证	test_duplicate_values_with_large_window	重复值数组与大窗口, 期望 [3,3]
40	补充验证	test_valid_input_enters_normal_calculation	条件合法, 期望正常计算 [8,5]
41	补充验证	test_oversized_window_raises_value_error	窗口越界, 期望 ValueError
42	补充验证	test_window_moves_across_array	窗口连续右移, 期望 [4,4,6]
43	辅助检查	test_translation_relation_with_added_constant	所有元素加常数, 输出同步增加
44	辅助检查	test_positive_scale_relation	所有元素乘正数, 输出同比例放大

表 17: 单元测试用例表 (三): 白盒测试、基本路径、循环测试与 Mock 测试

序号	测试方法	unittest 方法	输入或操作、预期结果
45	白盒覆盖	test_monotonic_queue_expires_front_candidate	单调队列队首过期, 期望 [9,2,3]
46	白盒覆盖	test_monotonic_queue_pops_smaller_tail	单调队列队尾弹出, 期望 [2,3,4]
47	白盒覆盖	test_monotonic_queue_keeps_larger_tail	单调队列队尾保留, 期望 [4,3,2]
48	白盒覆盖	test_monotonic_queue_handles_equal_values	相等元素条件组合, 期望 [2,2,2]
49	基本路径测试	test_basic_path_window_forms_immediately	第一次循环即形成窗口, 期望 [6]
50	基本路径测试	test_basic_path_window_waits_before_output	窗口尚未形成时不追加结果, 期望 [6]
51	基本路径测试	test_basic_path_expires_front_candidate	队首候选过期, 期望 [9,2,3]
52	基本路径测试	test_basic_path_pops_tail_candidate	队尾候选被弹出, 期望 [2,3,4]
53	基本路径测试	test_basic_path_keeps_tail_candidate	队尾候选被保留, 期望 [4,3,2]
54	循环测试	test_algorithm_loop_executes_once	单调队列主循环执行一次, 期望 [6]
55	循环测试	test_algorithm_loop_executes_twice	单调队列主循环执行两次, 期望 [6,1]
56	循环测试	test_algorithm_loop_executes_many_times	单调队列主循环执行多次, 期望 [4,7,7,3]
57	循环测试	test_nested_loop_repeated_tail_pops	嵌套循环连续弹出队尾, 期望 [3,4]
58	循环测试	test_concatenated_loops_expire_then_pop	串接循环先移除队首再弹出队尾, 期望 [9,10]

序号	测试方法	unittest 方法	输入或操作、预期结果
59	Mock 测试	<code>test_run_demo_prints_required_sample_result</code>	捕获示例输出，期望包含三种算法结果
60	Mock 测试	<code>test_run_demo_print_order_with_mock</code>	断言打印调用顺序与参数
61	Mock 测试	<code>test_run_demo_uses_mocked_algorithm_results</code>	控制算法返回值，断言算法调用参数与输出内容
62	入口测试	<code>test_main_returns_success_status</code>	调用 <code>main([])</code> ，期望返回 0
63	入口测试	<code>test_main_accepts_unused_arguments</code>	调用 <code>main(["unused"])</code> ，期望返回 0

8.11 测试执行结果与覆盖率分析

测试执行命令如下：

```
python -m unittest -v
```

测试输出摘要如下：

Listing 25: unittest 测试输出摘要

```
test_maximum_at_window_left_boundary ... ok
test_maximum_at_window_right_boundary ... ok
test_result_length_follows_formula ... ok
test_single_element_array ... ok
test_window_size_equals_array_length ... ok
test_window_size_is_max_minus ... ok
test_window_size_is_nominal_value ... ok
test_window_size_is_one ... ok
...
test_monotonic_queue_handles_equal_values ... ok
test_monotonic_queue_keeps_larger_tail ... ok
test_monotonic_queue_pops_smaller_tail ... ok
```

```
Ran 63 tests in 0.002s
```

```
OK
```

本次共运行 63 个测试方法，全部通过，测试通过率为：

$$\frac{63}{63} \times 100\% = 100\%$$

覆盖率统计使用 `coverage.py` 工具完成，命令如下：

```
python -m coverage run --branch --source=sliding_window_max -m unittest
python -m coverage report -m
```

实际覆盖率输出如下：

Listing 26: 语句覆盖率与分支覆盖率统计

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
------	-------	------	--------	--------	-------	---------

sliding_window_max.py	69	0	32	0	100%
TOTAL	69	0	32	0	100%

覆盖率结果显示，核心源码 `sliding_window_max.py` 中 69 条统计语句均被执行，未覆盖语句数为 0；32 个统计分支均被覆盖，未出现部分覆盖分支，总覆盖率为 100%。需要注意的是，覆盖率数据只能说明语句和分支被执行过，不能证明测试已经穷尽所有可能输入。因此，本实验没有单纯依赖覆盖率数字，而是结合等价类划分、边界值分析、健壮性测试、错误推测法、控制流路径分析、循环测试、辅助交叉检查和 Mock 测试共同判断测试充分性。

8.12 缺陷分析与测试结论

测试过程中未发现三种算法在 63 个测试方法上的输出错误。题目样例、负数数组、重复最大值数组、最小窗口、最大窗口、补充复杂输入和窗口移动输入均得到预期结果；所有非法输入均在输入校验阶段抛出预期异常；示例入口输出也通过 Mock 测试得到验证。

从测试设计角度看，程序运行时的主要风险和对应的处理情况如下：

- `bool` 类型在 Python 中属于 `int` 子类，容易被误判为合法整数；本实验通过错误推测法设计布尔值元素用例，并在校验函数中显式排除。
- 单调队列算法在重复最大值场景下可能出现候选下标维护错误；本实验使用重复最大值数组覆盖该风险。
- 只验证单一算法可能无法发现优化版本语义偏差；本实验通过 `subTest` 将三种算法绑定到同一组期望结果，实现交叉检查。
- 示例入口包含打印行为，普通返回值断言无法直接检查；本实验使用 `patch("builtins.print")` 捕获输出并断言样例结果。

综上，我们的测试集覆盖了模块接口、局部数据结构、边界条件、独立路径和出错处理五类单元测试内容；黑盒测试覆盖有效等价类、无效等价类、边界值、健壮性和错误推测，辅助检查覆盖变形关系和交叉验证；白盒测试覆盖主要控制流路径、循环路径和六种覆盖标准中的核心要求。测试通过率为 100%，代码覆盖率为 100%，说明当前测试集能够基本覆盖本实验核心代码的语句和分支。

9 实验总结

在本次实验中，我完成了滑动窗口最大值问题的三种实现，并围绕软件工程要求进行了规范检查、异常处理、性能分析和单元测试。暴力遍历版便于理解与验证，单调队列版体现了利用数据结构消除重复计算的优化思路，数组模拟队列版则进一步考察同一算法思想下的数据结构常数开销。在相同输入下，三种算法输出一致；在固定规模性能实验中，单调队列系列实现运行时间明显短于暴力版，符合从 $O(nk)$ 优化到 $O(n)$ 的复杂度变化。数组模拟队列版相对 `deque` 版只有小幅提升，说明主要性能问题已经由算法层级优化解决。

而从软件设计角度来看，将输入校验、核心算法、测试和性能分析拆分到不同职责模块，也有助于提高代码的可读性和可扩展性。后续若要继续完善，可以增加命令行参数解析、支持更多窗口统计指标，或将算法封装为可复用的小型 python 工具包。